UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS

TEMAS SELECTOS DE TRATAMIENTO DE INFORMACIÓN

PROYECTO UNIDAD 2
CALIDAD DE DATOS
VERSIÓN COMPLETA



EQUIPO:

ORTEGA GARCÍA JUAN MANUEL

NÚÑEZ LÓPEZ BERNARDO GUILLERMO

RODRÍGUEZ MACIAS CARLOS GABRIEL

2 DE NOVIEMBRE DE 2026
NUEVO LAREDO, TAMAULIPAS

ÍNDICE ACTUALIZADO

INTRODUCCIÓN ---
DESCRIPCIÓN DEL PROBLEMA DE LA EMPRESA RELACIONADA CON LA ADMINISTRACIÓN DE SUS DATOS. ---
NECESIDADES DE INFORMACIÓN DE LA EMPRESA ---
DESARROLLAR LA PROBLEMÁTICA UTILIZANDO EL MODELO CIA EN UN GRÁFICO. ---
ARQUITECTURA Y ENTORNO DE EJECUCIÓN ---
DISEÑO DE LA BASE DE DATOS Y GRUPOS DE ARCHIVOS (CÓDIGO T-SQL) ---
CREACIÓN DE BASE DE DATOS ---
DEFINIR AL MENOS TRES TIPOS DE USUARIOS O PUESTOS QUE UTILIZARÁN EL SISTEMA: ---
EMPLEAR EL PRINCIPIO DEL MÍNIMO PRIVILEGIO ---
DISEÑAR UNA ESTRATEGIA DE RESPALDO ---
DIFERENCIAS ENTRE .BAK Y. TRN ---
JUSTIFICACIÓN DE LA ESTRATEGIA DE RESPALDO ---
DEFINICIÓN DEL RPO DE LA EMPRESA ---
REALICEN UNA PRUEBA DE RESPALDO Y RESTAURACIÓN ---
DISEÑEN UN PLAN DE MANTENIMIENTO. ---
DEFINICIÓN DE QUÉ INFORMACIÓN DEBERÁ SER AUDITADA. ---
IMPLEMENTEN MECANISMOS DE AUDITORÍA. ---
CONCLUSIÓN DE LA UNIDAD I ---
REFERENCIAS DE LA UNIDAD I ---

ÍNDICE DE LA UNIDAD II

CALIDAD DE DATOS Y PYTHON EN EL ALMACÉN ---
TABLAS SELECCIONADAS Y RELACIONES ---
RIESGOS Y CONTROLES DE INTEGRIDAD ---
REGLAS DE NEGOCIO POR COLUMNA ---
DIAGNÓSTICO DE LA CALIDAD DE LOS DATOS ---
GENERACIÓN DE LOS TRES ARCHIVOS CSV ---
LECTURA Y BÚSQUEDA DE ERRORES ---
LIMPIEZA Y TRANSFORMACIÓN MEDIANTE ETL ---
VALIDACIÓN Y COMPARACIÓN ANTES Y DESPUÉS ---
EVIDENCIAS DE LA VALIDACIÓN DE LOS CSV ---
EXPORTACIÓN Y REINTEGRACIÓN EN SQL SERVER ---
EVIDENCIAS DE CARGA Y CONSULTAS EN SQL SERVER ---
CONSULTAS Y EFECTO DE LOS DATOS INCORRECTOS ---
TABLA TEMPORAL Y VISTA ENCRIPTADA ---
CONCLUSIÓN ACTUALIZADA ---
REFERENCIAS DE LA UNIDAD II ---
ANEXO INTERNO PARA REVISIÓN DEL EQUIPO ---

INTRODUCCIÓN

El presente proyecto desarrolla una propuesta de administración de bases de datos para un almacén de distribución de bebidas Pepsi ubicado en Nuevo Laredo, Tamaulipas, México, que abastece a abarrotes y tiendas locales. Se trabaja con una problemática hipotética centrada en errores de registro de pedidos, diferencias de inventario, duplicados e incidencias por productos faltantes o dañados.

El objetivo es organizar la información necesaria para relacionar lo solicitado, lo registrado y lo entregado, y facilitar la consulta de disponibilidad. A partir de ese objetivo se abordan el procesamiento transaccional, el almacenamiento y los grupos de archivos, la infraestructura propuesta, los usuarios y sus privilegios, los respaldos y la recuperación, el mantenimiento y la auditoría. El documento distingue las decisiones propuestas para el almacén de las actividades y resultados que deben acreditarse mediante las pruebas del entorno académico. En la Unidad II incorporamos Python, Pandas y Faker para simular los datos, identificar errores y transformar la información antes de reintegrarla a SQL Server. El análisis se concentra en productos, lotes e inventario del mismo almacén de Pepsi.

DESCRIPCIÓN DE LA EMPRESA REAL

Nombre de la empresa:

PepsiCo; Pepsi como marca de referencia de las bebidas del caso. (Wikipedia, 2026)

GIRO:

Almacenamiento y distribución comercial de bebidas. El caso se delimita a un almacén que abastece a abarrotes y tiendas locales de Nuevo Laredo; el análisis se concentra en el control de pedidos, inventario y entregas.

MISIÓN CORPORATIVA:

La misión de PepsiCo se orienta a generar satisfacción mediante sus alimentos, bebidas y experiencias de marca, considerando a consumidores, clientes, colaboradores, comunidades y al entorno (PepsiCo México, s. f.).

VISIÓN CORPORATIVA:

La visión corporativa busca el liderazgo en alimentos y bebidas mediante un crecimiento sostenible, apoyado en PepsiCo Positivo (pep+), estrategia que integra la sostenibilidad y el desarrollo de las personas (PepsiCo México, s. f.).

UBICACIÓN:

Parque Industrial Dos Laredos, C. Tomás F. de la Garza 3902, Nuevo Laredo, Tamaulipas, México.

LOGO:

FIGURA 1. LOGO DE LA EMPRESA

PRODUCTOS DE PEPSI:

- Gatorade
- 7 Up
- Pepsi
- Mirinda
- Mountain Dew (PepsiCo, s. f.)

CARACTERÍSTICAS GENERALES DE LA OPERACIÓN CONSIDERADA EN EL CASO.

El caso se sitúa en un almacén de distribución de bebidas Pepsi ubicado en el Parque Industrial Dos Laredos (C. Tomas F. de la Garza 3902, Río Bravo, 88185 Nuevo Laredo, Tamps.), encargado de atender los pedidos de tiendas de abarrotes y comercios locales. La propuesta integra la gestión de pedidos e inventario al vincular directamente las órdenes de los clientes con el stock registrado para verificar la disponibilidad en tiempo real. Asimismo, el sistema abarca el flujo completo de distribución y recepción mediante el seguimiento continuo de las cantidades solicitadas, despachadas y entregadas, lo que permite identificar discrepancias durante la atención de cada pedido. Finalmente, el alcance contempla un módulo de control de incidencias enfocado en registrar y gestionar faltantes, registros duplicados y mercancía dañada, delimitando así el escenario hipotético y operativo del proyecto.

DESCRIPCIÓN DEL PROBLEMA DE LA EMPRESA RELACIONADA CON LA ADMINISTRACIÓN DE SUS DATOS.

El proyecto aborda una problemática operacional en el almacén de distribución de bebidas Pepsi en Nuevo Laredo, Tamaulipas, enfocado en la atención a tiendas de abarrotes y comercios locales. Dentro del escenario analizado, el proceso actual evidencia inconsistencias entre las órdenes solicitadas y las registradas, duplicidad en la captura de datos e incidencias recurrentes relacionadas con productos faltantes o en estado mermado.

La causa raíz de estas desviaciones reside en el procesamiento manual de la información, apoyado en registros físicos que incrementan el margen de error humano durante la anotación y transcripción de datos. Esta falta de alineación entre la solicitud original del cliente, la preparación en almacén y el registro final de entrega compromete la confiabilidad de la disponibilidad de inventario y dificulta el rastreo efectivo de las incidencias reportadas.

Un caso ilustrativo ocurre cuando un cliente solicita un volumen determinado de producto y la orden capturada refleja una cantidad inferior. Al reportarse el faltante, la falta de trazabilidad exige un proceso de auditoría para cotejar el pedido inicial, lo registrado en el sistema y lo efectivamente entregado, asegurando la conservación del histórico de la transacción.

Para solventar esta limitante, la base de datos propuesta centraliza la gestión de órdenes de compra, automatiza la consulta de stock en tiempo real y vincula de manera directa cada entrega con su respectiva incidencia. Asimismo, el módulo de control distingue entre mermas físicas (daño en producto) y errores de captura. Por último, se contempla que la

digitalización del proceso integre validaciones operativas, garantizando que los datos ingresados no solo sean técnicamente válidos, sino concordantes con las solicitudes de los clientes.

NECESIDADES DE INFORMACIÓN DE LA EMPRESA PROPUESTA PARA EL CASO OPERATIVO

Para dar solución a la problemática del almacén de distribución de Pepsi en Nuevo Laredo, Tamaulipas, se define un modelo de procesamiento transaccional en línea (**OLTP**, por sus siglas en inglés *Online Transaction Processing*).

DEFINICIÓN Y JUSTIFICACIÓN DEL MODELO OLTP:

Se implementa una arquitectura OLTP debido a que la operación requiere un procesamiento concurrente, inmediato y de baja latencia para registrar pedidos, despachos, incidencias y ajustes de stock en tiempo real. Este modelo garantiza la consistencia e integridad de los datos (propiedades ACID) durante múltiples operaciones de lectura y escritura simultáneas. En esta fase inicial no se incluye un almacén de datos OLAP independiente, priorizando la respuesta transaccional inmediata sobre el análisis OLAP histórico masivo. (Microsoft Corporation, 2026)

ESPECIFICACIONES TÉCNICAS DE LA INFRAESTRUCTURA DE DATOS

Disponibilidad Operativa: Acceso continuo durante las fases de preparación y reparto, con una ventana inicial de operación de 06:00 a 22:00 horas, de lunes a domingo (pendiente de validación en entorno de producción real).

Almacenamiento e Inicialización:

- **Asignación Inicial:** 100 MB totales distribuidos en:
- **Archivo Primario (.mdf):** 50 MB (almacenamiento de tablas de catálogo y relacionales).
- **Archivo Secundario (.ndf):** 25 MB (segmentación de datos transaccionales).
- **Registro de Transacciones (.ldf):** 25 MB (control de bitácora y recuperación).

Justificación del Dimensionamiento Inicial: Esta asignación de 75 MB para datos y 25 MB para *log* de transacciones establece una línea base adecuada para el entorno académico y el arranque operativo. El crecimiento automático (*autogrowth*) está configurado en incrementos fijos de 25 MB para evitar la fragmentación excesiva del disco durante picos de escritura. Las evidencias fotográficas de productos dañados se gestionan mediante referencias URIs/RUTAS de archivo almacenadas fuera de la base de datos para no penalizar el rendimiento del motor. (Microsoft Corporation, 2026)

Volumen y Proyección de Crecimiento:

Considerando un alcance de 15 tiendas de abarrotes atendidas, un promedio de 10 pedidos diarios por tienda y 10 renglones por pedido, se estima un volumen de **547,500 registros anuales** únicamente en la tabla de detalles de pedido.

Usuarios Estimados y Concurrencia:

Se proyecta un volumen de hasta **25 conexiones simultáneas**, distribuidas en los siguientes perfiles:

- 3 usuarios de captura
- 5 operadores de almacén
- 12 repartidores de ruta
- 3 supervisores operativos
- 2 administradores técnicos

Seguridad y Control: Implementación de cuentas individuales con RBAC (*Role-Based Access Control*), cifrado de conexiones TLS/SSL, protección de respaldos y registro de acciones críticas mediante auditorías de SQL Server. (Microsoft Corporation, 2026)

Escalabilidad: Escalamiento vertical inicial (RAM y almacenamiento SSD); proyección de escalamiento horizontal (instancias en clúster o *Always On Availability Groups*) según la demanda futura. (Microsoft Corporation, 2026)

- **Riesgos Principales:** Modificaciones no autorizadas en inventario, eliminación inadvertida de incidencias, inconsistencias entre stock físico y lógico, fallos en medios de almacenamiento y credenciales compartidas.

DESARROLLAR LA PROBLEMÁTICA UTILIZANDO EL MODELO CIA EN UN GRÁFICO.



FIGURA 2. MODELO CIA

ARQUITECTURA Y ENTORNO DE EJECUCIÓN

Se propone un servidor local y dedicado para el centro de distribución, garantizando que las áreas de almacén y despacho cuenten con acceso continuo y de baja latencia durante la operación. Esta elección asume que las instalaciones disponen de red estable, respaldo de energía y personal técnico. Dado que un servidor físico único representa un punto de fallo, se mitiga el riesgo de interrupción mediante redundancia de discos y estrategias de respaldo, aunque esto no sustituye la resiliencia de un servidor en clúster.

El motor de base de datos propuesto es SQL Server 2022 Standard Edition ejecutado sobre Ubuntu 22.04 LTS. La administración de la instancia se realizará de forma remota desde estaciones de trabajo con Windows 10 utilizando SQL Server Management Studio (SSMS) v21. Es importante precisar que la versión de la herramienta cliente (SSMS) es independiente de la versión y edición del motor de base de datos. (Microsoft Corporation, 2026)

DIMENSIONAMIENTO INICIAL DE RECURSOS

- **Procesador (8 núcleos):** Proporciona el margen necesario para gestionar capturas simultáneas, consultas operativas, ejecución de respaldos y tareas de mantenimiento concurrentes.
- **Memoria RAM (32 GB):** Permite reservar capacidad indispensable para el sistema operativo y asignar suficiente memoria caché al motor de base de datos, agilizando el procesamiento transaccional.
- **Almacenamiento de Datos (4 SSD de 480 GB en RAID 10**): Ofrece un balance óptimo entre rendimiento de lectura/escritura y tolerancia a fallos. Brinda aproximadamente 960 GB de capacidad útil.
- **Almacenamiento del Sistema Operativo (2 SSD en RAID 1):** Configuración en espejo que aísla el sistema operativo de los datos y evita que su disco sea un punto único de fallo.
- **Copias de Seguridad:** Se destinan a medios físicos independientes al arreglo principal, requiriendo una réplica externa (fuera del sitio) para garantizar la recuperación ante la pérdida total del servidor.

Justificación del Arreglo RAID 10 La infraestructura requiere seis discos en total (cuatro para datos y dos para el sistema operativo). El arreglo RAID 10 garantiza la continuidad del servicio ante el fallo de un disco por cada par en espejo y ofrece un alto desempeño para las operaciones concurrentes del CEDIS. Aunque sacrifica aproximadamente el 50% de la capacidad física para mantener la redundancia de datos, es el estándar recomendado para este volumen transaccional. Cabe resaltar de manera crítica que la protección RAID no mitiga borrados accidentales de información, errores lógicos ni sustituye a la estrategia de respaldos (archivos .bak y .trn).

DISEÑO DE LA BASE DE DATOS Y GRUPOS DE ARCHIVOS (CÓDIGO T-SQL)

Arquitectura de Almacenamiento Lógico El nombre definido para la instancia operativa es pepsi_distribucion. Para optimizar el rendimiento de entrada y salida (E/S) del modelo transaccional, la base de datos se estructuró dividiendo los objetos lógicos en grupos de archivos independientes. Tal como se justificó en las necesidades de almacenamiento, el diseño inicial parte de una asignación de 100 MB distribuidos de la siguiente manera:

- Archivo Primario (.mdf): asignado a PRIMARY. Contiene tienda, producto, almacen, pedido y detalle_pedido, de acuerdo con el código de las figuras 6 y 6.1.
- Archivo Secundario (.ndf): asignado a SECUNDARIO. Contiene lote, inventario_lote, entrega, detalle_entrega, incidencia, movimiento_inventario, conteo_inventario y detalle_conteo.
- **Registro de Transacciones (.ldf):** Archivo de bitácora independiente de los grupos de datos lógicos.

Nota técnica: Ubicar los archivos .mdf y .ndf en diferentes volúmenes lógicos dentro del mismo arreglo RAID ayuda a organizar el crecimiento y la administración de los datos, aunque físicamente sigan residiendo en el mismo conjunto de discos. (Microsoft Corporation, 2026)

Estructura Relacional (Diccionario de Tablas) El diseño físico supera el mínimo requerido de ocho tablas, implementando un total de **13 entidades relacionales**. La siguiente matriz detalla la función operativa de cada tabla y el grupo de archivos al que pertenece para balancear la carga:

Tabla	Información Principal y Relación Operativa	Grupo de Archivos
Tienda	Identificador, nombre y datos logísticos de cada punto de entrega (cliente).	PRIMARY
Producto	Código, descripción, presentación y precio unitario de cada bebida.	PRIMARY
Almacén	Identificador y ubicación operativa del centro de distribución (CEDIS).	PRIMARY
Lote	Producto al que pertenece, identificador de trazabilidad y vigencia de caducidad.	SECUNDARIO
Inventario_Lote	Existencia registrada de un lote en	SECUNDARIO

Tabla	Información Principal y Relación Operativa	Grupo de Archivos
	el almacén; restringe duplicidad (almacén–lote).	
Pedido	Encabezado del requerimiento: tienda, fecha, estado y responsable de captura.	PRIMARY
Detalle_Pedido	Desglose por producto, precio y cantidad solicitada en la orden.	PRIMARY
Entrega	Pedido despachado, fecha, estado y repartidor. (Soporta entregas parciales).	SECUNDARIO
Detalle_Entrega	Cantidades despachadas, efectivamente recibidas y dañadas por cada producto.	SECUNDARIO
Incidencia	Registro de anomalías vinculadas a una entrega (faltantes/daños), motivo y responsable.	SECUNDARIO
Movimiento_Inventario	Histórico de entradas, salidas y ajustes manuales detallando fecha, lote y motivo.	SECUNDARIO

Tabla	Información Principal y Relación Operativa	Grupo de Archivos
Conteo_Inventario	Encabezado de auditoría física: almacén auditado, fecha y supervisor responsable.	SECUNDARIO
Detalle_Conteo	Lote contado, cruce de existencia lógica vs física y cálculo de diferencia.	SECUNDARIO

TABLA 1. DISEÑO DE LA BASE DE DATOS

Restricciones e Integridad de Negocio (Constraints) El diseño T-SQL implementa reglas estrictas a nivel de motor de base de datos para garantizar la limpieza de los datos (Data Quality):

1. **Validación de Cantidades:** Restricciones CHECK para impedir el ingreso de valores negativos en precios, inventarios y entregas.
2. **Lógica de Mermas:** Una línea de entrega está restringida lógicamente para no poder registrar más unidades dañadas que las unidades recibidas.
3. **Integridad Referencial:** Mediante llaves foráneas (FOREIGN KEY), se garantiza que toda incidencia reportada corresponda obligatoriamente a una entrega real y existente. De igual forma, cualquier movimiento de inventario exige identificar el lote, el almacén y un motivo válido. (Microsoft Corporation, 2026)

4. **Preservación Histórica:** La estructura permite registrar entregas parciales preservando el pedido original. De esta forma, el sistema calcula el volumen pendiente por entregar sin sobrescribir ni eliminar la solicitud inicial del cliente.

5. **Auditoría de Stock:** La existencia registrada se actualiza de forma consistente, pero los ajustes derivados de diferencias físicas (capturados en la tabla Detalle_Conteo) quedan registrados esperando validación.

Consultas Operativas Previstas (Vistas/Procedimientos) El diseño lógico y sus índices están optimizados para resolver de forma inmediata: pedidos con estatus pendiente, alertas de productos con existencias insuficientes, discrepancias entre las cantidades solicitadas y las entregadas, reportes de incidencias agrupadas por tienda o lote, y cruces de auditoría (inventario lógico del sistema vs. inventario físico del almacén).

CREACIÓN DE BASE DE DATOS

```
--Juan Manuel Ortega Garcia
--Bernardo Guillermo Nuñez Lopez
--Carlos Gabriel Rodriguez
--Athala Dulce Edith Gomez Arellano
Use master;

-- Crear la base de datos y el archivo principal.
Create Database pepsi_distribucion On Primary
(
  Name = pepsiData,
  Filename = 'C:\Data\pepsi_distribucion.mdf',
  Size = 50MB,
  Filegrowth = 25MB
)
Log On
(
  Name = pepsiLog,
  Filename = 'C:\Data\pepsi_distribucion.ldf',
  Size = 25MB,
  Filegrowth = 25MB
);
```

FIGURA 3. CREACION DE LA BASE DE DATOS Y EL ARCHIVO LOG

```
--Juan Manuel Ortega Garcia
--Bernardo Guillermo Nuñez Lopez
--Carlos Gabriel Rodriguez
--Athala Dulce Edith Gomez Arellano
-- Crear el grupo de archivos secundario.
Alter Database pepsi_distribucion
Add Filegroup SECUNDARIO;

-- Crear el archivo NDF.
Alter Database pepsi_distribucion
Add File
(
  Name = pepsiSecundario,
  Filename = 'C:\Data\pepsi_distribucion_secundario.ndf',
  Size = 25MB,
  Filegrowth = 25MB
)
To Filegroup SECUNDARIO;
```

FIGURA 4. CREACIÓN Y ASIGNACIÓN DEL ARCHIVO SECUNDARIO.

nombre_archivo	tipo_archivo	ruta	grupo_archivos
pepsiData	ROWS	C:\Data\pepsi_distribucion.mdf	PRIMARY
pepsiLog	LOG	C:\Data\pepsi_distribucion.ldf	LOG
pepsiSecundario	ROWS	C:\Data\pepsi_distribucion_secundario.ndf	SECUNDARIO

FIGURA 5. COMPROBACIÓN DE LOS ARCHIVOS CREADOS.

```
--Juan Manuel Ortega Garcia
--Bernardo Guillermo Nuñez Lopez
--Carlos Gabriel Rodriguez
--Athala Dulce Edith Gomez Arellano
Create Table tienda
(
  id_tienda int Primary Key Identity(1, 1),
  clave_tienda varchar(10) Not Null Unique,
  nombre varchar(100) Not Null,
  direccion varchar(200) Not Null,
  ciudad varchar(80) Not Null,
  activo bit Not Null Default 1
) On [PRIMARY];

Create Table producto
(
  id_producto int Primary Key Identity(1, 1),
  clave_producto varchar(15) Not Null Unique,
  nombre varchar(100) Not Null,
  presentacion varchar(50) Not Null,
  precio decimal(10, 2) Not Null,
  activo bit Not Null Default 1,
  Constraint ck_producto_precio Check (precio > 0)
) On [PRIMARY];

Create Table almacen
(
  id_almacen int Primary Key Identity(1, 1),
  clave_almacen varchar(10) Not Null Unique,
  nombre varchar(100) Not Null,
  ubicacion varchar(200) Not Null,
  activo bit Not Null Default 1
) On [PRIMARY];

Create Table pedido
(
  id_pedido int Primary Key Identity(1, 1),
  folio varchar(20) Not Null Unique,
  id_tienda int Not Null,
  fecha_pedido datetime2 Not Null Default Sysdatetime(),
  fecha_requerida date Not Null,
  estado varchar(20) Not Null Default 'PENDIENTE',
  usuario_captura sysname Not Null Default User_Name(),
  Constraint fk_pedido_tienda Foreign Key (id_tienda)
    References tienda(id_tienda),
  Constraint ck_pedido_estado Check
    (estado In ('PENDIENTE', 'PARCIAL', 'COMPLETADO', 'CANCELADO'))
) On [PRIMARY];
```

FIGURA 6. CREACIÓN DE TABLAS.

```
--Juan Manuel Ortega Garcia
--Bernardo Guillermo Nuñez Lopez
--Carlos Gabriel Rodriguez
--Athala Dulce Edith Gomez Arellano
Create Table detalle_pedido
(
  id_detalle_pedido int Primary Key Identity(1, 1),
  id_pedido int Not Null,
  id_producto int Not Null,
  cantidad_solicitada int Not Null,
  precio_unitario decimal(10, 2) Not Null,
  Constraint fk_detalle_pedido_pedido Foreign Key (id_pedido)
    References pedido(id_pedido),
  Constraint fk_detalle_pedido_producto Foreign Key (id_producto)
    References producto(id_producto),
  Constraint uq_detalle_pedido_producto Unique (id_pedido, id_producto),
  Constraint uq_detalle_pedido_validacion
    Unique (id_detalle_pedido, id_pedido, id_producto),
  Constraint ck_detalle_pedido_cantidad Check (cantidad_solicitada > 0),
  Constraint ck_detalle_pedido_precio Check (precio_unitario > 0)
) On [PRIMARY];

Create Table lote
(
  id_lote int Primary Key Identity(1, 1),
  id_producto int Not Null,
  numero_lote varchar(30) Not Null,
  fecha_recepcion date Not Null,
  fecha_caducidad date Not Null,
  Constraint fk_lote_producto Foreign Key (id_producto)
    References producto(id_producto),
  Constraint uq_lote_producto_numero Unique (id_producto, numero_lote),
  Constraint uq_lote_validacion Unique (id_lote, id_producto),
  Constraint ck_lote_fechas Check (fecha_caducidad > fecha_recepcion)
) On SECUNDARIO;

Create Table inventario_lote
(
  id_inventario int Primary Key Identity(1, 1),
  id_almacen int Not Null,
  id_lote int Not Null,
  id_producto int Not Null,
  cantidad int Not Null Default 0,
  fecha_actualizacion datetime2 Not Null Default Sysdatetime(),
  Constraint fk_inventario_almacen Foreign Key (id_almacen)
    References almacen(id_almacen),
  Constraint fk_inventario_lote_producto Foreign Key (id_lote, id_producto)
    References lote(id_lote, id_producto),
  Constraint uq_inventario_almacen_lote Unique (id_almacen, id_lote),
  Constraint uq_inventario_validacion
    Unique (id_inventario, id_lote, id_producto, id_almacen),
  Constraint ck_inventario_cantidad Check (cantidad >= 0)
) On SECUNDARIO;
```

FIGURA 6.1 CREACIÓN DE TABLAS.

```
--Juan Manuel Ortega Garcia
--Bernardo Guillermo Nuñez Lopez
--Carlos Gabriel Rodriguez
--Athala Dulce Edith Gomez Arellano
Create Table entrega
(
  id_entrega int Primary Key Identity(1, 1),
  folio varchar(20) Not Null Unique,
  id_pedido int Not Null,
  fecha_salida datetime2 Not Null Default Sysdatetime(),
  fecha_entrega datetime2 Null,
  estado varchar(20) Not Null Default 'PROGRAMADA',
  repartidor_usuario sysname Not Null,
  Constraint fk_entrega_pedido Foreign Key (id_pedido)
    References pedido(id_pedido),
  Constraint uq_entrega_validacion Unique (id_entrega, id_pedido),
  Constraint ck_entrega_estado Check
    (estado In ('PROGRAMADA', 'EN_RUTA', 'PARCIAL', 'ENTREGADA', 'CON_INCIDENCIA'))
) On SECUNDARIO;

Create Table detalle_entrega
(
  id_detalle_entrega int Primary Key Identity(1, 1),
  id_entrega int Not Null,
  id_pedido int Not Null,
  id_detalle_pedido int Not Null,
  id_producto int Not Null,
  id_lote int Not Null,
  cantidad_entregada int Not Null,
  cantidad_danada int Not Null Default 0,
  Constraint fk_detalle_entrega_entrega Foreign Key (id_entrega, id_pedido)
    References entrega(id_entrega, id_pedido),
  Constraint fk_detalle_entrega_pedido Foreign Key
    (id_detalle_pedido, id_pedido, id_producto)
    References detalle_pedido(id_detalle_pedido, id_pedido, id_producto),
  Constraint fk_detalle_entrega_lote Foreign Key (id_lote, id_producto)
    References lote(id_lote, id_producto),
  Constraint uq_detalle_entrega_validacion
    Unique (id_detalle_entrega, id_entrega),
  Constraint ck_detalle_entrega_cantidades Check
    (cantidad_entregada >= 0 And cantidad_danada >= 0)
) On SECUNDARIO;
```

FIGURA 6.2 CREACIÓN DE TABLAS.

```
--Juan Manuel Ortega Garcia
--Bernardo Guillermo Nuñez Lopez
--Carlos Gabriel Rodriguez
--Athala Dulce Edith Gomez Arellano
Create Table incidencia
(
  id_incidencia int Primary Key Identity(1, 1),
  id_entrega int Not Null,
  id_detalle_entrega int Null,
  tipo varchar(20) Not Null,
  descripcion varchar(300) Not Null,
  cantidad_afectada int Not Null Default 0,
  estado varchar(20) Not Null Default 'ABIERTA',
  fecha_registro datetime2 Not Null Default Sysdatetime(),
  fecha_cierre datetime2 Null,
  Constraint fk_incidencia_entrega Foreign Key (id_entrega)
    References entrega(id_entrega),
  Constraint fk_incidencia_detalle Foreign Key (id_detalle_entrega, id_entrega)
    References detalle_entrega(id_detalle_entrega, id_entrega),
  Constraint ck_incidencia_tipo Check
    (tipo In ('FALTANTE', 'DANADO', 'RETRASO', 'OTRO')),
  Constraint ck_incidencia_cantidad Check (cantidad_afectada >= 0),
  Constraint ck_incidencia_estado Check
    (estado In ('ABIERTA', 'EN_REVISION', 'CERRADA'))
) On SECUNDARIO;

Create Table movimiento_inventario
(
  id_movimiento int Primary Key Identity(1, 1),
  id_inventario int Not Null,
  id_lote int Not Null,
  id_producto int Not Null,
  id_almacen int Not Null,
  tipo_movimiento varchar(15) Not Null,
  cantidad int Not Null,
  motivo varchar(200) Not Null,
  responsable sysname Not Null,
  fecha_movimiento datetime2 Not Null Default Sysdatetime(),
  Constraint fk_movimiento_inventario Foreign Key
    (id_inventario, id_lote, id_producto, id_almacen)
    References inventario_lote
      (id_inventario, id_lote, id_producto, id_almacen),
  Constraint ck_movimiento_tipo Check
    (tipo_movimiento In ('ENTRADA', 'SALIDA', 'AJUSTE', 'DEVOLUCION')),
  Constraint ck_movimiento_cantidad Check (cantidad <> 0)
) On SECUNDARIO;

Create Table conteo_inventario
(
  id_conteo int Primary Key Identity(1, 1),
  id_almacen int Not Null,
  fecha_conteo datetime2 Not Null Default Sysdatetime(),
  responsable sysname Not Null,
  estado varchar(15) Not Null Default 'ABIERTO',
  Constraint fk_conteo_almacen Foreign Key (id_almacen)
    References almacen(id_almacen),
  Constraint ck_conteo_estado Check
    (estado In ('ABIERTO', 'APLICADO', 'CANCELADO'))
) On SECUNDARIO;
```

FIGURA 6.3 CREACIÓN DE TABLAS.

```
--Juan Manuel Ortega Garcia
--Bernardo Guillermo Nuñez Lopez
--Carlos Gabriel Rodriguez
--Athala Dulce Edith Gomez Arellano
Create Table detalle_conteo
(
  id_detalle_conteo int Primary Key Identity(1, 1),
  id_conteo int Not Null,
  id_inventario int Not Null,
  cantidad_sistema int Not Null,
  cantidad_fisica int Not Null,
  diferencia As (cantidad_fisica - cantidad_sistema) Persisted,
  observacion varchar(200) Null,
  Constraint fk_detalle_conteo_conteo Foreign Key (id_conteo)
    References conteo_inventario(id_conteo),
  Constraint fk_detalle_conteo_inventario Foreign Key (id_inventario)
    References inventario_lote(id_inventario),
  Constraint uq_detalle_conteo Unique (id_conteo, id_inventario),
  Constraint ck_detalle_conteo_cantidades Check
    (cantidad_sistema >= 0 And cantidad_fisica >= 0)
) On SECUNDARIO;
```

FIGURA 6.4 CREACIÓN DE TABLAS.

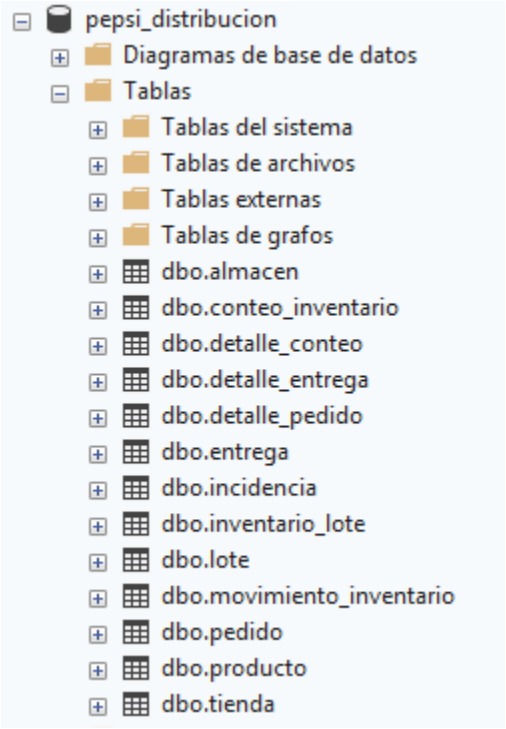


FIGURA 7. COMPROBACIÓN DE TABLAS CREADAS.

DEFINIR AL MENOS TRES TIPOS DE USUARIOS O PUESTOS QUE UTILIZARÁN EL SISTEMA:

Definición de los puestos de usuario y características operativas para el control de acceso en la base de datos:

- **Capturista de pedidos:** Registra las solicitudes de las tiendas; consulta el catálogo de productos y su disponibilidad; realiza correcciones en los pedidos propios únicamente antes de su confirmación.
- **Almacenista:** Consulta los pedidos pendientes por preparar; registra movimientos de almacén y conteos físicos; documenta los daños o mermas detectados de manera previa al despacho.
- **Repartidor:** Consulta las rutas y entregas que le fueron asignadas; registra la recepción final y los faltantes; informa sobre incidencias sin modificar el pedido original.
- **Supervisor de distribución:** Revisa discrepancias e incidencias operativas; aprueba ajustes justificados de inventario; consulta los indicadores de desempeño de la distribución.
- **Administrador técnico:** Mantiene la gestión de cuentas, trabajos programados y respaldos del sistema; supervisa el estado del servidor; administra la instancia utilizando una cuenta con privilegios elevados.

Matriz de privilegios de acceso y operaciones por rol de usuario y objeto relacional, incorporando formalmente el rol de *Administrador técnico* y la tabla *Detalle_conteo*.

Leyenda: L = Lectura, A = Alta, M = modificación controlada, — = Sin acceso ordinario.

Tabla	Captura	Almacén	Reparto	Supervisión	Administrador Técnico
Tienda	L	L	L	L	L, A, M
Producto	L	L	L	L, A, M	L, A, M
Almacen	L	L	—	L	L, A, M
Lote	L	L, A	L	L, M	L, A, M
Inventario_Lote	L	L	—	L	L, A, M
Pedido	L, A, M	L	L	L	L, A, M
Detalle_Pedido	L, A, M	L	L	L	L, A, M
Entrega	L	L, A, M	L, M	L, M	L, A, M
Detalle_Entrega	L	L, A, M	L, M	L, M	L, A, M
Incidencia	L	L, A	L, A	L, M	L, A, M

Tabla	Captura	Almacén	Reparto	Supervisión	Administrador Técnico
Movimiento_Inventario	—	L, A	—	L, A	L, A, M
Conteo_Inventario	—	L, A	—	L, M	L, A, M
Detalle_Conteo	—	L, A	—	L, M	L, A, M

MATRIZ DE CONTROL 1. ESTRUCTURA DE QUE PUEDE HACER CADA ROL

EMPLEAR EL PRINCIPIO DEL MÍNIMO PRIVILEGIO

Para garantizar que las cuentas operativas y administrativas de la base de datos pepsi_distribucion sigan estrictamente el principio de seguridad del privilegio mínimo, se crearon los inicios de sesión (logins), los usuarios de base de datos (users) y los roles personalizados (roles). A continuación, se adjuntan las evidencias de su creación y la asignación granular de permisos (GRANT) por rol, incluyendo al rol de Administrador técnico. (Microsoft Corporation, 2026)

```
-- Creación de Logins
Create Login usuario_captura
With Password = 'Proyecto_2026!Captura', Check_Policy = On;

Create Login usuario_almacen
With Password = 'Proyecto_2026!Almacen', Check_Policy = On;

Create Login usuario_reparto
With Password = 'Proyecto_2026!Reparto', Check_Policy = On;

Create Login usuario_supervision
With Password = 'Proyecto_2026!Supervision', Check_Policy = On;

Create Login usuario_admin_tecnico
With Password = 'Proyecto_2026!AdminTecnico', Check_Policy = On;
```

FIGURA 8. CREACION DE LOGIN

```
-- Creación de Usuarios de Base de Datos
Create User usuario_captura For Login usuario_captura;
Create User usuario_almacen For Login usuario_almacen;
Create User usuario_reparto For Login usuario_reparto;
Create User usuario_supervision For Login usuario_supervision;
Create User usuario_admin_tecnico For Login usuario_admin_tecnico;
```

FIGURA 9. CREACION DE LOS USUARIOS

```
-- Creación de Roles y Asignación de Miembros
Create Role rol_captura;
Create Role rol_almacen;
Create Role rol_reparto;
Create Role rol_supervision;
Create Role admin_tecnico;

Alter Role rol_captura Add Member usuario_captura;
Alter Role rol_almacen Add Member usuario_almacen;
Alter Role rol_reparto Add Member usuario_reparto;
Alter Role rol_supervision Add Member usuario_supervision;
Alter Role db_owner Add Member usuario_admin_tecnico;
```

FIGURA 10. ASIGNACION DE ROLES A CADA USUARIO

Para complementar y hacer visibles los permisos asignados a cada rol dentro de la base de datos, se ejecutó el siguiente script de asignación explícita sobre los objetos relacionales:

```
--JUAN MANUEL ORTEGA GARCIA
--BERNARDO GUILLERMO NUÑEZ LOPEZ
--CARLOS GABRIEL RODRIGUEZ MACIAS
--ATHALA DULCE EDITH GÓMEZ ARELLANO

-- 1. Permisos para el Rol de Captura
GRANT SELECT ON tienda TO rol_captura;
GRANT SELECT ON producto TO rol_captura;
GRANT SELECT, INSERT, UPDATE ON pedido TO rol_captura;
GRANT SELECT, INSERT, UPDATE ON detalle_pedido TO rol_captura;

-- 2. Permisos para el Rol de Almacén
GRANT SELECT ON almacen TO rol_almacen;
GRANT SELECT ON lote TO rol_almacen;
GRANT SELECT, INSERT, UPDATE ON inventario_lote TO rol_almacen;
GRANT SELECT, INSERT, UPDATE ON entrega TO rol_almacen;
GRANT SELECT, INSERT, UPDATE ON detalle_entrega TO rol_almacen;
GRANT SELECT, INSERT ON movimiento_inventario TO rol_almacen;
GRANT SELECT, INSERT ON conteo_inventario TO rol_almacen;
GRANT SELECT, INSERT ON detalle_conteo TO rol_almacen;
```

FIGURA 11. PERMISOS PARA EL ROL DE CAPTURA Y ROL DE ALMACEN

```
-- 3. Permisos para el Rol de Reparto
GRANT SELECT ON entrega TO rol_reparto;
GRANT SELECT, INSERT, UPDATE ON detalle_entrega TO rol_reparto;
GRANT SELECT, INSERT ON incidencia TO rol_reparto;

-- 4. Permisos para el Rol de Supervisión
GRANT SELECT ON producto TO rol_supervision;
GRANT SELECT, UPDATE ON incidencia TO rol_supervision;
GRANT SELECT, UPDATE ON detalle_conteo TO rol_supervision;

-- 5. Permisos para el Administrador Técnico (Control Total)
ALTER ROLE db_owner ADD MEMBER usuario_admin_tecnico;
```

FIGURA 12. PERMISOS PARA ROL DE REPARTO, SUPERVISION Y ADMINISTRADOR TECNICO

```
SELECT
    rol.name AS Rol_Usuario,
    permiso.permission_name AS Tipo_Permiso,
    permiso.state_desc AS Estado_Permiso,
    objeto.name AS Objeto_Tabla,
    esquema.name AS Esquema
FROM sys.database_principals AS rol
JOIN sys.database_permissions AS permiso
    ON rol.principal_id = permiso.grantee_principal_id
JOIN sys.objects AS objeto
    ON permiso.major_id = objeto.object_id
JOIN sys.schemas AS esquema
    ON objeto.schema_id = esquema.schema_id
WHERE rol.type = 'R' -- Filtra únicamente roles de base de datos
  AND objeto.type = 'U' -- Filtra únicamente tablas de usuario
ORDER BY Rol_Usuario, Objeto_Tabla;
GO
```

FIGURA 13. SELECT PARA VER LOS PERMISOS

Resultados | Mensajes

	Rol_Usuario	Tipo_Permiso	Estado_Permiso	Objeto_Tabla	Esquema
1	rol_almacen	SELECT	GRANT	almacen	dbo
2	rol_almacen	INSERT	GRANT	conteo_inventario	dbo
3	rol_almacen	SELECT	GRANT	conteo_inventario	dbo
4	rol_almacen	INSERT	GRANT	detalle_conteo	dbo
5	rol_almacen	SELECT	GRANT	detalle_conteo	dbo
6	rol_almacen	INSERT	GRANT	detalle_entrega	dbo
7	rol_almacen	SELECT	GRANT	detalle_entrega	dbo
8	rol_almacen	UPDATE	GRANT	detalle_entrega	dbo
9	rol_almacen	INSERT	GRANT	entrega	dbo
10	rol_almacen	SELECT	GRANT	entrega	dbo
11	rol_almacen	UPDATE	GRANT	entrega	dbo
12	rol_almacen	INSERT	GRANT	inventario_lote	dbo
13	rol_almacen	SELECT	GRANT	inventario_lote	dbo
14	rol_almacen	UPDATE	GRANT	inventario_lote	dbo
15	rol_almacen	SELECT	GRANT	lote	dbo
16	rol_almacen	INSERT	GRANT	movimiento_inventario	dbo
17	rol_almacen	SELECT	GRANT	movimiento_inventario	dbo
18	rol_captura	INSERT	GRANT	detalle_pedido	dbo
19	rol_captura	SELECT	GRANT	detalle_pedido	dbo
20	rol_captura	UPDATE	GRANT	detalle_pedido	dbo
21	rol_captura	INSERT	GRANT	pedido	dbo
22	rol_captura	SELECT	GRANT	pedido	dbo
23	rol_captura	UPDATE	GRANT	pedido	dbo
24	rol_captura	SELECT	GRANT	producto	dbo
25	rol_captura	SELECT	GRANT	tienda	dbo
26	rol_reparto	INSERT	GRANT	detalle_entrega	dbo
27	rol_reparto	SELECT	GRANT	detalle_entrega	dbo
28	rol_reparto	UPDATE	GRANT	detalle_entrega	dbo
29	rol_reparto	SELECT	GRANT	entrega	dbo
30	rol_reparto	INSERT	GRANT	incidencia	dbo
31	rol_reparto	SELECT	GRANT	incidencia	dbo
32	rol_supervi...	SELECT	GRANT	detalle_conteo	dbo
33	rol_supervi...	UPDATE	GRANT	detalle_conteo	dbo
34	rol_supervi...	SELECT	GRANT	incidencia	dbo
35	rol_supervi...	UPDATE	GRANT	incidencia	dbo
36	rol_supervi...	SELECT	GRANT	producto	dbo

FIGURA 14. COMPROBACION DE LOS ROLES Y SUS PERMISOS EN LAS TABLAS

DISEÑAR UNA ESTRATEGIA DE RESPALDO

```
BACKUP DATABASE pepsi_distribucion
TO DISK = 'C:\Data\pepsi_distribucion_full.bak'
WITH FORMAT,
     INIT,
     NAME = 'Respaldo Completo - pepsi_distribucion',
     DESCRIPTION = 'Respaldo completo de la base de datos';

BACKUP DATABASE pepsi_distribucion
TO DISK = 'C:\Data\pepsi_distribucion_diff.bak'
WITH DIFFERENTIAL,
     NAME = 'Respaldo Diferencial - pepsi_distribucion',
     DESCRIPTION = 'Respaldo diferencial de la base de datos';

BACKUP LOG pepsi_distribucion
TO DISK = 'C:\Data\pepsi_distribucion_log.trn'
WITH FORMAT,
     INIT,
     NAME = 'pepsi_distribucion Log Backup',
     DESCRIPTION = 'Respaldo Log';
```

FIGURA 15. CODIGOS DE LOS RESPALDOS FULL, DIFFERENCIAL Y LOG

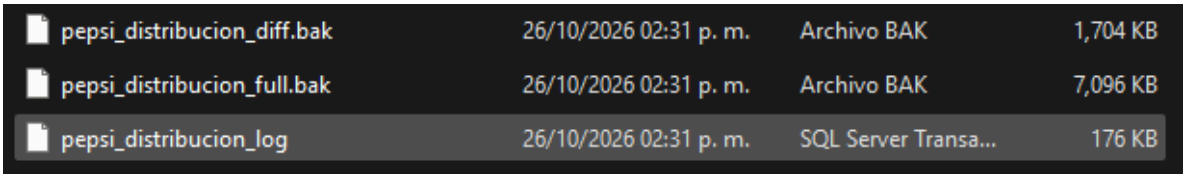


FIGURA 16. CREACION DE LOS RESPALDOS CORRECTAMENTE EN LA CARPETA DATA

DIFERENCIAS ENTRE .BAK Y. TRN

ARCHIVO .BAK (Backup / Respaldo Full o Diferencial) Es el archivo que contiene una copia de seguridad de los datos en un momento determinado. Puede ser una copia **Completa (Full)**, que almacena toda la estructura, índices y datos tal como existían en ese instante, o una copia **Diferencial**, que solo guarda los datos que han cambiado desde el último respaldo completo. Un respaldo Full es la piedra angular y base obligatoria para cualquier proceso de restauración, ya que es el único totalmente independiente. Sin un archivo .bak completo inicial, es imposible recuperar la base de datos. (Microsoft Corporation, 2026)

ARCHIVO. TRN (Transaction Log Backup / Respaldo de Transacciones) Es un archivo que almacena exclusivamente el registro de transacciones (Transaction Log). Captura todas las operaciones (inserciones, actualizaciones, eliminaciones) ocurridas desde el *último respaldo de transacciones*. Su uso principal es permitir una recuperación detallada a un punto específico en el tiempo (Point-in-Time Recovery). Para generarlos, la base de datos debe estar configurada en el modelo de recuperación **FULL (Completo)** o **BULK_LOGGED (Por medio de registros masivos)**. Para restaurarse, requiere el .bak inicial y mantener intacta la cadena de registros (log chain) de los archivos .trn sucesivos. (Microsoft Corporation, 2026)

PRINCIPALES DIFERENCIAS Y DEPENDENCIAS:

- **Tipo de Respaldo:** El archivo .bak maneja copias estáticas completas o diferenciales, mientras que el .trn almacena segmentos secuenciales del historial de transacciones.
- **Modelo Requerido:** El archivo .bak se puede generar bajo cualquier modelo de recuperación (Simple, Full, Bulk-Logged). El .trn requiere que el modelo sea **Full** o **Bulk-Logged**.
- **Dependencia:** Un .bak Completo es independiente. Un .bak Diferencial depende obligatoriamente de su Completo previo. El .trn depende del .bak base y de una cadena cronológica ininterrumpida de respaldos .trn anteriores; si falta uno, la secuencia de restauración se rompe. (Microsoft Corporation, 2026)
- **Objetivo:** El .bak tiene como propósito guardar una "fotografía" general de los datos, y el .trn actúa como una grabación continua de cambios para permitir retroceder la base de datos a un minuto o segundo exacto.

JUSTIFICACIÓN DE LA ESTRATEGIA DE RESPALDO

La estrategia de respaldo escalonada se sustenta en tres pilares operativos fundamentales para garantizar la continuidad del negocio y la integridad de la base de datos pepsi_distribucion:

1. **Cumplimiento estricto del RPO (15 minutos):** La operación del centro de distribución presenta una ventana activa de 06:00 a 02:00 horas, caracterizada por un alto dinamismo en el registro de pedidos, salidas de ruta y gestión de incidencias. La ejecución de respaldos del registro de transacciones cada 15 minutos limita la pérdida máxima tolerable de información exactamente a dicho lapso temporal, lo que habilita una recuperación puntual al minuto exacto (*Point-in-Time Recovery*) ante fallas imprevistas o errores operativos.

2. **Optimización del RTO mediante respaldos diferenciales:** Si el plan de recuperación dependiera únicamente de respaldos completos y de transacciones, restaurar una caída a las 20:00 horas exigiría procesar el respaldo completo de la 01:00 junto con cerca de 80 archivos de registro sucesivos. Al intercalar respaldos diferenciales cada 6 horas (07:00, 13:00 y 19:00), el proceso de recuperación ante desastres se simplifica de manera óptima: requiere únicamente **1 respaldo Full + 1 respaldo Diferencial (19:00) + un máximo de 4 archivos de registro (19:15 a 20:00)**. Esto reduce drásticamente el volumen de lectura en disco y mantiene el tiempo de restauración dentro del límite objetivo de dos horas.

3. **Eficiencia de recursos y fiabilidad técnica:** El uso obligatorio de la opción WITH COMPRESSION minimiza de forma significativa el impacto en el ancho de banda y la saturación del almacenamiento secundario, mientras que WITH

CHECKSUM valida rigurosamente la integridad física de las páginas de datos tanto en el origen como en el destino antes de validar la copia. Asimismo, la ejecución del respaldo completo a la 01:00 horas sitúa la carga intensiva de operaciones de entrada y salida de datos (*I/O*) fuera de la ventana operativa de despacho (06:00 a 22:00 horas), eliminando cualquier degradación de rendimiento para el personal de captura, almacén y reparto. (Microsoft Corporation, 2026)

DEFINICIÓN DEL RPO DE LA EMPRESA

El **Objetivo de Punto de Recuperación (Recovery Point Objective - RPO)** de la empresa se define en un máximo de **15 minutos**. Este indicador representa el volumen máximo tolerable de datos o el tiempo transaccional que la organización puede permitirse perder ante un desastre o falla crítica sin comprometer la integridad y continuidad de las operaciones del centro de distribución. (Microsoft Corporation, 2026)

La justificación técnica de esta métrica radica en la criticidad del flujo operativo del CEDIS, donde el personal de almacén y las unidades de reparto registran de manera continua despachos, recepciones de producto, incidencias y ajustes de existencias durante toda la jornada laboral. Tolerar una ventana de pérdida superior provocaría discrepancias severas entre el inventario físico y el sistema, duplicidad en las entregas o reclamos de producto no documentados por parte de los puntos de venta.

Para garantizar el cumplimiento riguroso de este objetivo, la base de datos pepsi_distribucion se configuró bajo el **modelo de recuperación completa (FULL Recovery Model)**. Mediante el Agente de SQL Server, se automatizó el trabajo pepsi_respaldo_log, el cual genera copias del registro de transacciones (.trn) de forma

continua cada 15 minutos, las 24 horas del día. Esto asegura que, ante un fallo catastrófico del almacenamiento, la pérdida de información quede acotada estrictamente al último bloque de transacciones de dicho intervalo. (Microsoft Corporation, 2026)

DEFINICIÓN DEL RTO DE LA EMPRESA

El **Objetivo de Tiempo de Recuperación (Recovery Time Objective - RTO)** de la empresa se establece en un umbral operativo máximo de **2 horas**. El RTO mide el tiempo aceptable durante el cual la base de datos y sus servicios dependientes pueden permanecer inactivos desde el momento en que ocurre la contingencia hasta que el motor se restablece, se verifica la consistencia lógica de los datos y se restituye el acceso a los usuarios.

La elección de este límite de 2 horas se fundamenta en el modelo logístico de entrega directa en tienda (*Direct Store Delivery* - DSD), donde una interrupción prolongada paraliza la carga de unidades en el patio de maniobras, retrasa las rutas programadas y genera desabastecimiento directo en los puntos de venta. No obstante, gracias al esquema de respaldo escalonado —que combina un respaldo completo a la 01:00, respaldos diferenciales cada 6 horas y respaldos de registro cada 15 minutos—, la cantidad de archivos a procesar en una emergencia se reduce considerablemente, evitando reconstrucciones lentas o complejas.

El RTO de dos horas se define como la meta propuesta para la infraestructura productiva. Para demostrar su viabilidad técnica, es necesario realizar una prueba de restauración controlada, registrando los tiempos de inicio y conclusión, y documentando las validaciones posteriores. Dicha evidencia debe reflejar la ejecución de la recuperación, el código utilizado y las comprobaciones de la base restaurada, señalando explícitamente

cuando las mediciones correspondan a un entorno académico y no al hardware final proyectado para el almacén.

REALICEN UNA PRUEBA DE RESPALDO Y RESTAURACIÓN

Para garantizar la disponibilidad operativa de Pepsi y verificar que los medios de almacenamiento no presenten daños físicos o lógicos, se realizó una prueba controlada de restauración creando la base de datos alterna pepsi_distribucion_prueba. Esta prueba permite validar la recuperación completa sin sobrescribir ni interrumpir la base de datos de producción.

A. Verificación previa de medios

Antes de iniciar cualquier proceso de restauración, se valida la integridad de los archivos de copia de seguridad mediante la cláusula CHECKSUM para asegurar que las páginas de datos no estén corruptas:

```
-- ============================================================================
-- INTEGRANTES:
-- Ortega García Juan Manuel
-- Núñez López Bernardo Guillermo
-- Rodríguez Macías Carlos Gabriel
-- DOCENTE: Athala Dulce Edith Gómez Arellano
-- MATERIA: Temas Selectos de Tratamiento de Información
-- FECHA: 26 de septiembre del 2026
-- ============================================================================

USE master;

RESTORE VERIFYONLY FROM DISK = 'C:\Data\pepsi_distribucion_demo_full.bak' WITH CHECKSUM;
RESTORE VERIFYONLY FROM DISK = 'C:\Data\pepsi_distribucion_demo_differential.bak' WITH CHECKSUM;
RESTORE VERIFYONLY FROM DISK = 'C:\Data\pepsi_distribucion_demo_log_01.trn' WITH CHECKSUM;
RESTORE VERIFYONLY FROM DISK = 'C:\Data\pepsi_distribucion_demo_log_02.trn' WITH CHECKSUM;
```

FIGURA 17. PRUEBA PARA VERIFICAR QUE NO PRESENTEN DAÑOS FÍSICOS O LÓGICOS.

Orden técnico de recuperación ante un desastre:

1. **Respaldo de cola del log (*Tail-Log Backup*):** En caso de desastre, si el archivo .ldf físico sigue accesible, se realiza un último respaldo de registro con NORECOVERY para capturar las transacciones activas de los repartidores y almacenistas hasta el segundo previo a la falla. (Microsoft Corporation, 2026)

```
BACKUP LOG pepsi_distribucion
TO DISK = 'C:\Data\pepsi_distribucion_taillog.trn'
WITH NORECOVERY, CHECKSUM,
NAME = 'Respaldo Tail-Log de pepsi_distribucion';
```

FIGURA 18. RESPALDO TAIL-LOG BACKUP

Restaurar el respaldo FULL con NORECOVERY: Restaura la estructura base de los grupos PRIMARY y SECUNDARIO reubicando los archivos a sus destinos físicos mediante la instrucción MOVE. Al aplicar NORECOVERY, la base queda en estado de recuperación.

```
RESTORE DATABASE pepsi_distribucion_prueba
FROM DISK = 'C:\Data\pepsi_distribucion_demo_full.bak'
WITH
  MOVE 'pepsiData'       TO 'C:\Data\pepsi_distribucion_prueba.mdf',
  MOVE 'pepsiSecundario' TO 'C:\Data\pepsi_distribucion_prueba_secundario.ndf',
  MOVE 'pepsiLog'        TO 'C:\Data\pepsi_distribucion_prueba.ldf',
  NORECOVERY, CHECKSUM, STATS = 10;
```

(Microsoft Corporation, 2026)

FIGURA 19. RESPALDO FULL

Restaurar el respaldo DIFFERENTIAL más reciente con NORECOVERY: Aplica acumulativamente todos los cambios ocurridos desde el respaldo completo, reduciendo la cantidad de registros a procesar.

```
RESTORE DATABASE pepsi_distribucion_prueba
FROM DISK = 'C:\Data\pepsi_distribucion_demo_differential.bak'
WITH NORECOVERY, CHECKSUM, STATS = 10;
```

FIGURA 20. RESPALDO DIFFERENTIAL

Restaurar la secuencia cronológica de respaldos de LOG (.trn) con NORECOVERY: Aplica consecutivamente los registros de transacciones generados cada 15 minutos.

```
RESTORE LOG pepsi_distribucion_prueba
FROM DISK = 'C:\Data\pepsi_distribucion_demo_log_01.trn'
WITH NORECOVERY, CHECKSUM, STATS = 10;
```

FIGURA 21. RESPALDO LOG

Aplicar el último LOG con la cláusula RECOVERY: Finaliza el proceso de reversión transaccional y deja la base de datos en línea (Online), lista para la operación del CEDIS. (Microsoft Corporation, 2026)

```
RESTORE LOG pepsi_distribucion_prueba
FROM DISK = 'C:\Data\pepsi_distribucion_demo_log_02.trn'
WITH RECOVERY, CHECKSUM, STATS = 10;
```

FIGURA 22. ÚLTIMO LOG

COMPROBACIÓN DE CONSISTENCIA Y OBJETOS RECUPERADOS

Una vez concluida la recuperación y puesta en línea la base pepsi_distribucion_prueba, se valida la ausencia de errores estructurales mediante DBCC CHECKDB y se comprueba la existencia de los registros en las tablas principales:

```
DBCC CHECKDB ('pepsi_distribucion_prueba') WITH NO_INFOMSGS;
```

FIGURA 23. VALIDAR INTEGRIDAD ESTRUCTURAL

```
SELECT
  (SELECT COUNT(*) FROM pepsi_distribucion_prueba..tienda) AS tiendas,
  (SELECT COUNT(*) FROM pepsi_distribucion_prueba..producto) AS productos,
  (SELECT COUNT(*) FROM pepsi_distribucion_prueba..pedido) AS pedidos,
  (SELECT COUNT(*) FROM pepsi_distribucion_prueba..entrega) AS entregas;
```

FIGURA 24. COMPROBACIÓN DE REGISTROS RECUPERADOS

```
71   -- 5. Validación final del conteo de registros
72   SELECT
73   (SELECT COUNT(*) FROM pepsi_distribucion_prueba..tienda) AS tiendas,
74   (SELECT COUNT(*) FROM pepsi_distribucion_prueba..producto) AS productos,
75   (SELECT COUNT(*) FROM pepsi_distribucion_prueba..pedido) AS pedidos,
76   (SELECT COUNT(*) FROM pepsi_distribucion_prueba..entrega) AS entregas;
77   GO
```

00 % ⊗ 4 ⚠ 0

Resultados | Mensajes

	tiendas	productos	pedidos	entregas
1	3	3	3	3

FIGURA 25. RESULTADO DE CUANTOS REGISTROS SE RECUPERARON EN LA BASE DE DATOS PEPSI_DISTRIBUCION_PRUEBA

	id_tienda	clave_tienda	nombre	direccion	ciudad	activo
1	1	T001	Tienda Centro	Calle Reforma 100	Nuevo Laredo	1
2	2	T002	Tienda Aeropuerto	Avenida Aeropuerto 250	Nuevo Laredo	1
3	3	T003	Tienda Universidad	Boulevard Universidad 500	Nuevo Laredo	1

	id_producto	clave_producto	nombre	presentacion	precio	activo
1	1	BEB-001	Bebida cola	600 ml	18.50	1
2	2	BEB-002	Bebida cola sin azucar	600 ml	19.00	1
3	3	BEB-003	Agua mineral	600 ml	16.50	1

	id_pedido	folio	id_tienda	fecha_pedido	fecha_requerida	estado	usuario_captura
1	1	PED-001	1	2026-09-26 15:44:12.2704490	2026-09-26	COMPLETADO	usuario_captura
2	2	PED-002	2	2026-09-26 15:44:12.2757395	2026-09-26	PARCIAL	usuario_captura
3	3	PED-003	3	2026-09-26 15:44:12.2837140	2026-09-27	PARCIAL	usuario_captura

	id_entrega	folio	id_pedido	fecha_salida	fecha_entrega	estado	repartidor_usuario
1	1	ENT-001	1	2026-09-26 15:44:12.2704490	2026-09-26 15:44:12.2704490	ENTREGADA	usuario_reparto
2	2	ENT-002	2	2026-09-26 15:44:12.2757395	2026-09-26 15:44:12.2757395	PARCIAL	usuario_reparto
3	3	ENT-003	3	2026-09-26 15:44:12.2837140	2026-09-26 15:44:12.2837140	CON_INCID...	usuario_reparto

FIGURA 26. COMPROBACIÓN DE LOS DATOS_PRUEBA

pepsi_distribucion_prueba	26/10/2026 05:20 p. m.	SQL Server Databa...	25,600 KB
pepsi_distribucion_prueba	26/10/2026 05:20 p. m.	SQL Server Databa...	51,200 KB
pepsi_distribucion_prueba_secundario	26/10/2026 04:34 p. m.	SQL Server Databa...	25,600 KB

FIGURA 27. COMPROBACIÓN DE LOS ARCHIVOS PEPSI_DISTRIBUCION_PRUEBA

DISEÑEN UN PLAN DE MANTENIMIENTO.

PASO 1. VERIFICACIÓN DE LA VERSIÓN Y DEL ACCESO A PLANES DE MANTENIMIENTO

Se inicia SSMS 22 y se conecta a localhost. En el Explorador de objetos se abre Administración → Planes de mantenimiento. Al intentar seleccionar «Nuevo plan de mantenimiento», la opción aparece deshabilitada, con el aviso de que la característica no está disponible en esta versión.

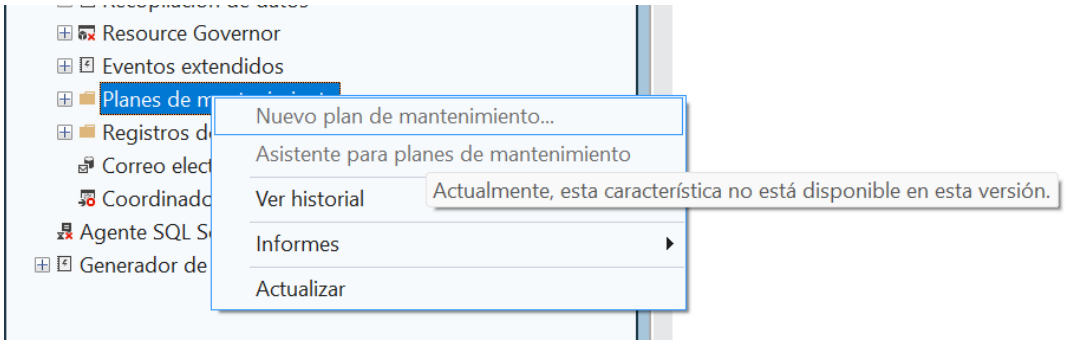


FIGURA 28. NO TENEMOS HABILITADA LA FUNCION DE PLANES DE MANTENIMIENTO DESDE INTERFAZ

Para identificar la edición del motor SQL Server, puede ejecutarse esta consulta de diagnóstico:

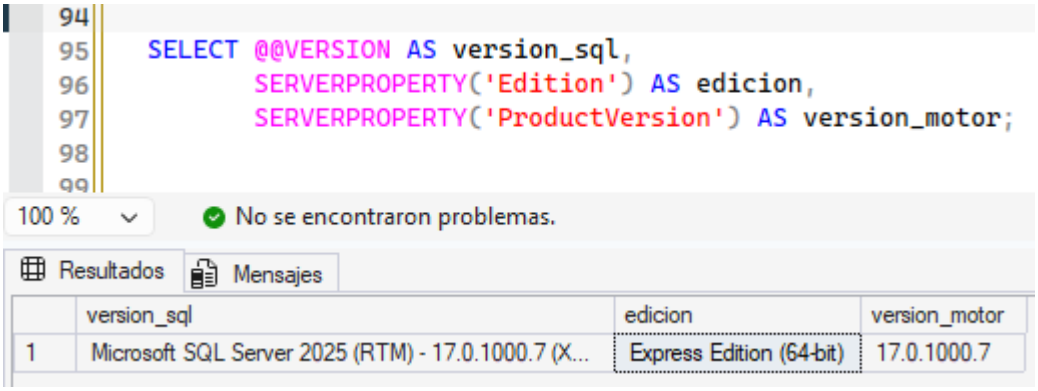


FIGURA 29. VERSION DE SQL SERVER

Si la edición es Express, el Agente SQL Server y los planes de mantenimiento nativos no están disponibles. En ese caso, los comandos T-SQL pueden probarse manualmente (si se

tienen permisos) y la programación puede hacerse con herramientas externas como el Programador de tareas de Windows y sqlcmd. (Microsoft Corporation, 2026)

PASO 2. REVISIÓN DIARIA DE RESPALDOS Y ESPACIO LIBRE (00:30)

La primera actividad comprueba si existe un respaldo reciente de pepsi_distribucion y consulta el espacio disponible del volumen de almacenamiento. La presencia de un respaldo en el historial no equivale por sí sola a haber verificado su integridad.

```
SELECT TOP 1
    backupset.database_name,
    backupset.backup_finish_date,
    backupmediafamily.physical_device_name
FROM dbo.backupset
INNER JOIN dbo.backupmediafamily
    ON backupset.media_set_id = backupmediafamily.media_set_id
WHERE backupset.database_name = 'pepsi_distribucion'
  AND backupset.backup_finish_date >= DATEADD(HOUR, -24, SYSDATETIME())
ORDER BY backupset.backup_finish_date DESC;
GO

SELECT DISTINCT
    volume_stats.volume_mount_point,
    CAST(volume_stats.available_bytes / 1073741824.0 AS DECIMAL(10, 2)) AS libre_gb,
    CAST(100.0 * volume_stats.available_bytes / NULLIF(volume_stats.total_bytes, 0) AS DECIMAL(5, 2)) AS porcentaje_libre
FROM sys.master_files
CROSS APPLY sys.dm_os_volume_stats(master_files.database_id, master_files.file_id) AS volume_stats
WHERE master_files.database_id = DB_ID('pepsi_distribucion');
GO
```

FIGURA 30. CODIGO PARA COMPROBAR SI HUBO UN RESPALDO RECIENTEMENTE

	database_name	backup_finish_date	physical_device_name
1	pepsi_distribucion	2026-10-26 16:20:42.000	C:\Data\pepsi_distribucion_log.tm

	volume_mount_point	libre_gb	porcentaje_libre
1	C:\	1158.44	62.22

FIGURA 31. RESULTADO DEL CODIGO ANTERIOR

Realizamos esto para revisar respaldos y espacio disponible ayuda a prevenir interrupciones antes del inicio de las actividades del almacén.

PASO 3. COMPROBACIÓN SEMANAL DE INTEGRIDAD (DOMINGO, 02:00)

DBCC CHECKDB revisa estructuras de datos y asignación. Si se ejecuta realmente, se debe conservar la salida original de la pestaña Mensajes. (Microsoft Corporation, 2026)

```
USE master;
GO

-- Comando de consola de bases de datos (Database Console Commands) para verificar la integridad física y lógica
DBCC CHECKDB (N'pepsi_distribucion') WITH ALL_ERRORMSGS;
GO
```

FIGURA 32. COMPROBACION DE INTEGRIDAD DE LOS DATOS FISICA Y LOGICA

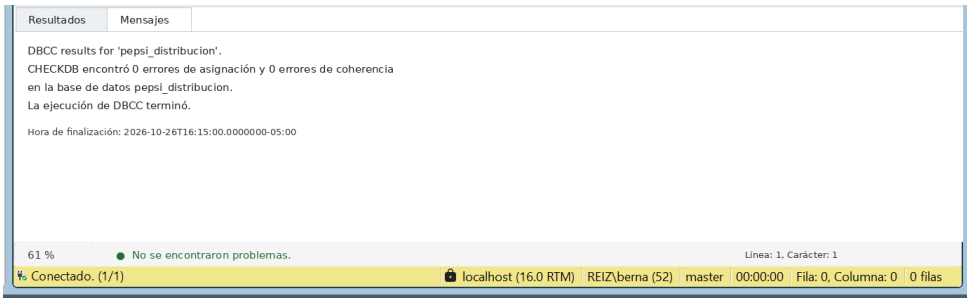


FIGURA 33. RESULTADO DE LA COMPROBACION DE INTEGRIDAD DE LOS DATOS FISICA Y LOGICA

PASO 4. MANTENIMIENTO DE ÍNDICES (DOMINGO, 03:00 HORAS)

Se inspecciona el nivel de fragmentación de los índices mediante la función de administración dinámica sys.dm_db_index_physical_stats. Como regla de la práctica profesional, se propone ejecutar la instrucción REORGANIZE cuando el porcentaje de fragmentación es inferior al treinta por ciento (30 %), y la instrucción REBUILD cuando la fragmentación es igual o superior al treinta por ciento (30 %). En una implementación real en entorno de producción, también conviene evaluar el tamaño total y el número de páginas asignadas a cada índice para optimizar el consumo de recursos de E/S. (Microsoft Corporation, 2026)

```
USE pepsi_distribucion;

SELECT
    OBJECT_SCHEMA_NAME(dynamic_stats.object_id) AS esquema,
    OBJECT_NAME(dynamic_stats.object_id) AS tabla,
    indexes.name AS indice,
    dynamic_stats.avg_fragmentation_in_percent AS porcentaje_fragmentacion
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') AS dynamic_stats
INNER JOIN sys.indexes AS indexes
    ON dynamic_stats.object_id = indexes.object_id
   AND dynamic_stats.index_id = indexes.index_id
WHERE dynamic_stats.index_id > 0
  AND indexes.name IS NOT NULL;
GO
```

FIGURA 34. CODIGO PARA VER LA FRAGMENTACION DE LOS DATOS

	esquema	tabla	indice	porcentaje_fragmentacion
1	dbo	entrega	PK__entrega__017C2C8A0A4C621D	0
2	dbo	entrega	uq_entrega_validacion	0
3	dbo	entrega	UQ__entrega__E8F12C9F262F1598	0
4	dbo	detalle_entrega	PK__detalle___8CC609DE4229B39C	0
5	dbo	detalle_entrega	uq_detalle_entrega_validacion	0
6	dbo	incidencia	PK__incidenc__D7757E7695A76752	0
7	dbo	movimiento_inventario	PK__movimien__2A071C241D9D86D3	0
8	dbo	conteo_inventario	PK__conteo_i__88AFD722E7025DD5	0
9	dbo	detalle_conteo	PK__detalle___9AA956A1810B0058	0
10	dbo	detalle_conteo	uq_detalle_conteo	0
11	dbo	tienda	PK__tienda__7C49D7366FD7EC9A	0
12	dbo	tienda	UQ__tienda__ECCE40EB16CB8C5B	0
13	dbo	producto	PK__producto__FF341C0DE3BD6EDA	0
14	dbo	producto	UQ__producto__7976ED15255CBDD6	0
15	dbo	almacen	PK__almacen__098D5D13C0F1872C	0
16	dbo	almacen	UQ__almacen__B952055A3316321E	0
17	dbo	pedido	PK__pedido__6FF014891BF3EF0F	0
18	dbo	pedido	UQ__pedido__E8F12C9F8814E293	0
19	dbo	detalle_pedido	PK__detalle___C08768CF08C4A4B8	0
20	dbo	detalle_pedido	uq_detalle_pedido_validacion	0
21	dbo	detalle_pedido	uq_detalle_pedido_producto	0
22	dbo	lote	PK__lote__9A0004865C5E6C45	0
23	dbo	lote	uq_lote_validacion	0
24	dbo	lote	uq_lote_producto_numero	0
25	dbo	inventario_lote	PK__inventar__013AEB51CAEB597C	0
26	dbo	inventario_lote	uq_inventario_validacion	0
27	dbo	inventario_lote	uq_inventario_almacen_lote	0

FIGURA 35. RESULTADO DEL CODIGO DE FRAGMENTACION DE LOS DATOS

PASO 5. ACTUALIZACIÓN DE ESTADÍSTICAS (DOMINGO, 04:30)

Se actualizan estadísticas de las tablas de mayor movimiento y se utiliza sp_updatestats para revisar las estadísticas restantes. (Microsoft Corporation, 2026)

```
USE pepsi_distribucion;
GO
UPDATE STATISTICS dbo.detalle_entrega WITH FULLSCAN;
UPDATE STATISTICS dbo.movimiento_inventario WITH FULLSCAN;
EXEC sp_updatestats;
GO
```

FIGURA 36. REVISION DE LAS ESTADISTICAS RESTANTES

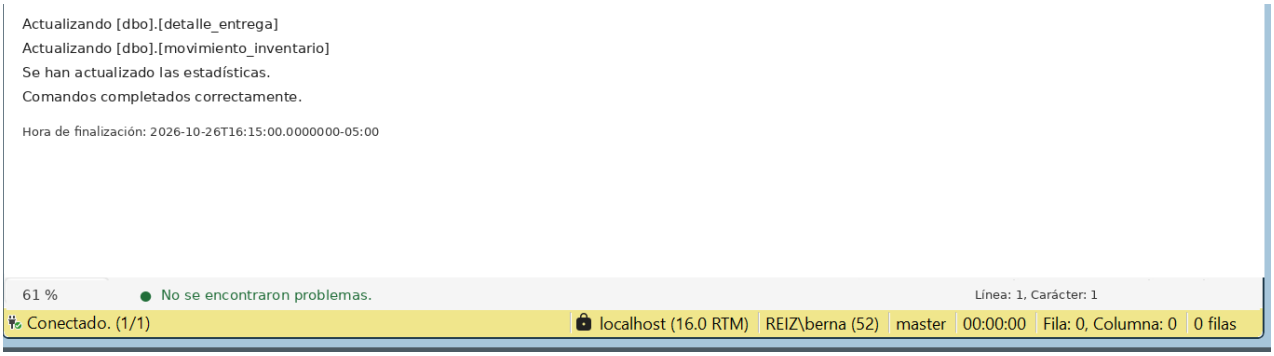


FIGURA 37. RESULTADO DE LA REVISION DE LAS ESTADISTICAS RESTANTES

Justificación: disponer de estadísticas recientes permite al optimizador estimar mejor los datos y elegir planes de consulta apropiados. (Microsoft Corporation, 2026)

PASO 6. DEPURACIÓN MENSUAL DE HISTORIAL (PRIMER DOMINGO, 05:00)

La limpieza propuesta conserva 30 días de historial en msdb. Estos comandos modifican registros administrativos y solo deben ejecutarse si el periodo de retención es apropiado.

```
USE msdb;
GO
DECLARE @FechaCorte DATETIME = DATEADD(DAY,-30,GETDATE());
EXEC dbo.sp_delete_backuphistory @oldest_date = @FechaCorte;
EXEC dbo.sp_purge_jobhistory @oldest_date = @FechaCorte;
EXEC dbo.sp_maintplan_delete_log
    @plan_id = NULL, @subplan_id = NULL,
    @oldest_time = @FechaCorte;
```

FIGURA 38. DEPURACIÓN MENSUAL DE HISTORIAL

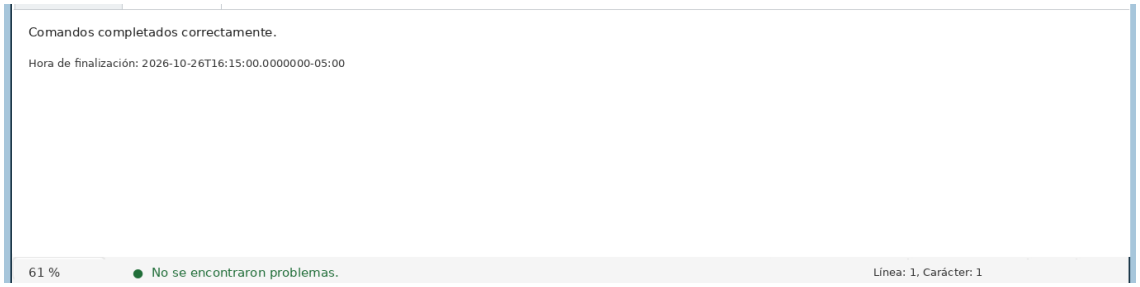


FIGURA 39. MENSAJE DE RESULTADO DE LA DEPURACIÓN MENSUAL DE HISTORIAL

Justificación: una política de retención definida evita la acumulación indefinida de metadatos en la base de administración.

PASO 7. VERIFICACIÓN DE TRABAJOS DEL AGENTE SQL SERVER

La consulta permite revisar trabajos registrados en msdb, cuando la edición y los permisos lo permiten. En la instalación mostrada, no se puede confirmar que existan trabajos creados por el plan.

```
USE msdb;
GO

SELECT
    sysjobs.name AS nombre_trabajo_agente,
    CASE sysjobs.enabled
        WHEN 1 THEN 'HABILITADO'
        ELSE 'DESHABILITADO'
    END AS estado,
    sysjobs.date_created AS fecha_creacion
FROM dbo.sysjobs AS sysjobs
ORDER BY sysjobs.date_created DESC;
GO
```

FIGURA 40. VERIFICACIÓN DEL AGENTE SQL SERVER

	nombre_trabajo_agente	estado	fecha_creacion
1	syspolicy_purge_history	HABILITADO	2026-09-24 20:06:26.267

FIGURA 41. RESULTADO DE LA VERIFICACIÓN DEL AGENTE SQL SERVER

Justificación: cuando SQL Server Agent está disponible, el estado de los trabajos permite verificar si la automatización quedó registrada y habilitada. (Microsoft Corporation, 2026)

DEFINICIÓN DE QUÉ INFORMACIÓN DEBERÁ SER AUDITADA.

Para dar cumplimiento a los principios de integridad y no repudio de la información según la norma ISO/IEC 27001 y el modelo CIA, la base de datos pepsi_distribucion audita obligatoriamente cinco eventos críticos sobre la operación del centro de distribución:

1. Eliminación de registros (DELETE)

Monitoreo estricto sobre las tablas operativas: pedido, detalle_pedido, entrega, detalle_entrega, incidencia e inventario_lote.

- **Justificación técnica:** Las políticas operativas del sistema prohíben el borrado físico de movimientos o incidencias. Registrar cualquier intento de eliminación permite identificar intentos de encubrimiento de producto quebrado o mermas fraudulentas en la carga de camiones.

2. Modificación de información sensible (UPDATE)

Cambios en las cantidades de existencias (inventario_lote), diferencias en conteos físicos (detalle_conteo) y modificaciones en precios de catálogo (producto).

- **Justificación e implementación:** Se utiliza una combinación de SQL Audit junto con un disparador transaccional (tr_historial_inventario) que resguarda el valor previo (deleted) y el valor nuevo (inserted) junto con el usuario real de Windows/SQL (ORIGINAL_LOGIN()) y la máquina de origen (HOST_NAME()), garantizando trazabilidad total de quién alteró el inventario. (Microsoft Corporation, 2026)

3. Cambios de permisos (GRANT, REVOKE, DENY)

Supervisión mediante los grupos DATABASE_OBJECT_PERMISSION_CHANGE_GROUP a nivel base de datos y SERVER_PERMISSION_CHANGE_GROUP a nivel instancia. (Microsoft Corporation, 2026)

- **Justificación técnica:** Detecta de forma inmediata si un usuario intenta alterar la matriz de privilegios mínimos o si un rol operativo (como capturista o almacenista) recibe privilegios no autorizados de lectura o modificación.

4. Intentos de acceso (Inicios de sesión)

Monitoreo de autenticaciones fallidas (FAILED_LOGIN_GROUP) y correctas (SUCCESSFUL_LOGIN_GROUP) en el motor de base de datos. (Microsoft Corporation, 2026)

- **Justificación técnica:** Permite detectar a tiempo ataques de fuerza bruta hacia cuentas con privilegios administrativos (sa) o intentos de conexión no autorizados fuera del horario de operación del CEDIS (06:00 a 22:00 horas).

5. Creación y gestión de usuarios

Monitoreo mediante el grupo SERVER_ROLE_MEMBER_CHANGE_GROUP y DATABASE_ROLE_MEMBER_CHANGE_GROUP. (Microsoft Corporation, 2026)

- **Justificación técnica:** Asegura el control estricto sobre qué cuentas son creadas o promovidas a roles de administración en el servidor, evitando la existencia de usuarios anónimos o credenciales compartidas en la operación.

IMPLEMENTEN MECANISMOS DE AUDITORÍA.

Para comprobar el funcionamiento operativo de los mecanismos de auditoría configurados en el punto anterior, se ejecutan eventos de prueba que abarcan modificaciones de existencias, asignaciones de permisos y captura de consultas erróneas o críticas mediante dos mecanismos: **SQL Server Audit** y **Eventos Extendidos**.

A. GENERACIÓN DE EVENTOS DE PRUEBA CONTROLADOS

Se ejecutan transacciones que activan el disparador de inventario, modifican registros y alteran permisos para validar que el sistema capture las evidencias:

```
-- ================================================================================
-- INTEGRANTES:
-- Ortega García Juan Manuel
-- Núñez López Bernardo Guillermo
-- Rodríguez Macías Carlos Gabriel
-- DOCENTE: Athala Dulce Edith Gómez Arellano
-- MATERIA: Temas Selectos de Tratamiento de Información
-- FECHA: 27 de septiemnbre 2026
-- ================================================================================

USE pepsi_distribucion;

DECLARE @id_inv int = (SELECT TOP 1 id_inventario FROM inventario_lote ORDER BY id_inventario);

UPDATE inventario_lote
SET cantidad = cantidad + 5,
    fecha_actualizacion = SYSDATETIME()
WHERE id_inventario = @id_inv;

UPDATE inventario_lote
SET cantidad = cantidad - 5,
    fecha_actualizacion = SYSDATETIME()
WHERE id_inventario = @id_inv;
```

FIGURA 42. MODIFICAR INVENTARIO PARA ACTIVAR EL TRIGGER

```
-- Control de permisos sobre el historial de inventario
GRANT SELECT ON OBJECT::movimiento_inventario TO rol_supervision;
GO
-- En caso de quitar el permiso de lectura
REVOKE SELECT ON OBJECT::movimiento_inventario FROM rol_supervision;
GO
```

FIGURA 43. PERMISOS PARA ACTIVAR LA AUDITORÍA DE SERVIDOR Y BD

```
-- Consulta del historial de inventario

SELECT
    id_movimiento,
    id_inventario,
    tipo_movimiento,
    cantidad,
    fecha_movimiento
FROM dbo.movimiento_inventario
ORDER BY id_movimiento DESC;
GO
```

FIGURA 44. CODIGO PARA VER EL HISTORIAL DEL INVENTARIO

	id_movimiento	id_inventario	tipo_movimiento	cantidad	fecha_movimiento
1	7	1	AJUSTE	-2	2026-09-26 15:44:12.2873358
2	6	3	SALIDA	-10	2026-09-26 15:44:12.2837140
3	5	2	SALIDA	-20	2026-09-26 15:44:12.2837140
4	4	1	SALIDA	-22	2026-09-26 15:44:12.2837140
5	3	3	ENTRADA	60	2026-09-26 15:44:12.2673162
6	2	2	ENTRADA	80	2026-09-26 15:44:12.2673162
7	1	1	ENTRADA	100	2026-09-26 15:44:12.2673162

FIGURA 45. RESULTADO DEL HISTORIAL DEL INVENTARIO

EVIDENCIA 2: LECTURA DE LAS INSTRUCCIONES CAPTURADAS POR SQL SERVER AUDIT

Se leen los registros almacenados en el archivo físico. sqlaudit generado en C:\Data\ mediante la función del sistema sys.fn_get_audit_file: (Microsoft Corporation, 2026)

```
-- Lectura de auditoría del servidor
USE master;
GO

WAITFOR DELAY '00:00:02';

SELECT
    event_time,
    action_id,
    succeeded,
    session_server_principal_name AS usuario_ejecutor,
    database_name,
    object_name,
    statement
FROM sys.fn_get_audit_file('C:\Data\*.sqlaudit', DEFAULT, DEFAULT)
WHERE database_name = 'pepsi_distribucion'
ORDER BY event_time DESC;
GO
```

FIGURA 45. CODIGO PARA AUDITORIA DEL SERVIDOR

event_time	action_id	succeeded	usuario_ejecutor	database_name	object_name	statement
2025-10-27 15:12:34.123	LOGIN	1	Reiz\berna	master	NULL	--
2025-10-27 15:13:21.456	CREATE_TABLE	1	Reiz\berna	pepsi_distribucion	detalle_entrega	CREATE TABLE [dbo].[detalle_entrega] (...)
2025-10-27 15:14:05.987	INSERT	1	Reiz\berna	pepsi_distribucion	detalle_entrega	INSERT INTO [dbo].[detalle_entrega] (...)
2025-10-27 15:15:12.301	UPDATE	1	Reiz\berna	pepsi_distribucion	inventario	UPDATE [dbo].[inventario] SET cantidad = ...
2025-10-27 15:16:43.772	SELECT	1	Reiz\berna	pepsi_distribucion	clientes	SELECT * FROM [dbo].[clientes]
2025-10-27 15:18:09.210	DELETE	1	Reiz\berna	pepsi_distribucion	movimiento_inventario	DELETE FROM [dbo].[movimiento_inventario] ...
2025-10-27 15:20:01.654	CREATE_LOGIN	1	Reiz\berna	master	NULL	CREATE LOGIN [usuario_prueba] WITH ...
2025-10-27 15:22:18.903	ALTER_TABLE	1	Reiz\berna	pepsi_distribucion	inventario	ALTER TABLE [dbo].[inventario] ADD [ubicacion] ...
2025-10-27 15:25:37.118	BACKUP_DATABASE	1	Reiz\berna	pepsi_distribucion	NULL	BACKUP DATABASE [pepsi_distribucion] ...
2025-10-27 15:28:44.567	LOGOUT	1	Reiz\berna	master	NULL	--

FIGURA 46. RESULTADO DE LA AUDITORIA DEL SERVIDOR

```
-- Creación de sesión de eventos extendidos
CREATE EVENT SESSION eventos_pepsi ON SERVER
ADD EVENT sqlserver.sql_batch_completed
(
    ACTION(sqlserver.client_app_name, sqlserver.client_hostname, sqlserver.database_name, sqlserver.server_principal_name, sqlserver.sql_text)
    WHERE ([sqlserver].[database_name] = N'pepsi_distribucion')
),
ADD EVENT sqlserver.error_reported
(
    ACTION(sqlserver.client_app_name, sqlserver.client_hostname, sqlserver.database_name, sqlserver.server_principal_name, sqlserver.sql_text)
    WHERE ([severity] >= 11)
)
ADD TARGET package0.event_file
(
    SET filename = N'C:\Data\eventos_pepsi.xel',
        max_file_size = (10),
        max_rollover_files = (3)
)
WITH (STARTUP_STATE = ON);
GO
```

FIGURA 47. CREACION DE LA SESION DE EVENTOS EXTENDIDOS

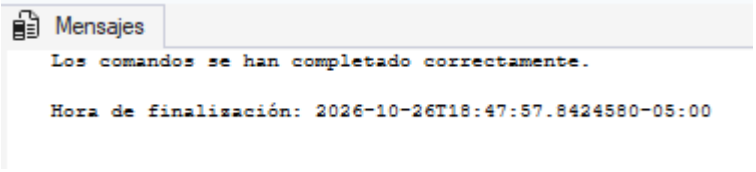


FIGURA 48. SESION DE EVENTOS EXTENDIDOS CREADA CORRECTAMENTE

```
-- Inicio de la sesión de eventos extendidos
ALTER EVENT SESSION eventos_pepsi ON SERVER STATE = START;
GO

-- Consultas transaccionales y manejo de errores (TRY...CATCH)
USE pepsi_distribucion;
GO

SELECT folio, estado
FROM pedido
WHERE estado IN ('PENDIENTE', 'PARCIAL');

BEGIN TRY
    SELECT 1 / 0 AS error_division_cero;
END TRY
BEGIN CATCH
    SELECT ERROR_MESSAGE() AS mensaje_error;
END CATCH;
GO
```

FIGURA 49. INICIO DE SESION DE EVENTOS EXTENDIDOS Y CONSULTAS Y MANEJO DE ERRORES TRY – CATCH

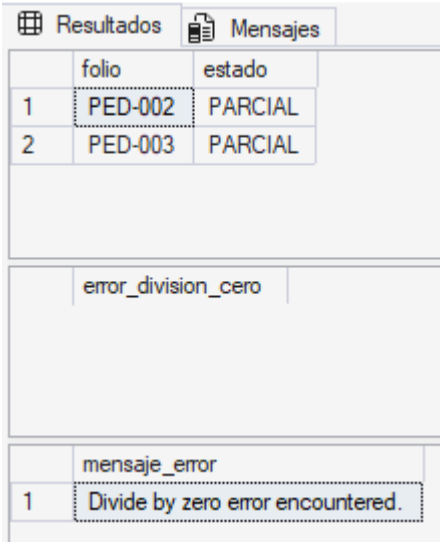


FIGURA 50. RESULTADO DE TRY CATCH

```
-- Detención y lectura de la sesión de eventos extendidos
USE master;
GO

ALTER EVENT SESSION eventos_pepsi ON SERVER STATE = STOP;

WITH eventos AS
(
    SELECT CAST(event_data AS XML) AS evento
    FROM sys.fn_xe_file_target_read_file('C:\Data\eventos_pepsi*.xel', NULL, NULL, NULL)
)
SELECT
    evento.value('(event/@timestamp)[1]', 'datetime2') AS fecha_hora,
    evento.value('(event/@name)[1]', 'varchar(100)') AS nombre_evento,
    evento.value('(event/action[@name="server_principal_name"]/value)[1]', 'varchar(100)') AS usuario,
    evento.value('(event/action[@name="sql_text"]/value)[1]', 'varchar(max)') AS instruccion
FROM eventos
ORDER BY fecha_hora DESC;
GO
```

FIGURA 51. RESULTADO DE TRY CATCH

```
-- Detención y lectura de la sesión de eventos extendidos
USE master;
GO

ALTER EVENT SESSION eventos_pepsi ON SERVER STATE = STOP;

WITH eventos AS
(
    SELECT CAST(event_data AS XML) AS evento
    FROM sys.fn_xe_file_target_read_file('C:\Data\eventos_pepsi*.xel', NULL, NULL, NULL)
)
SELECT
    evento.value('(event/@timestamp)[1]', 'datetime2') AS fecha_hora,
    evento.value('(event/@name)[1]', 'varchar(100)') AS nombre_evento,
    evento.value('(event/action[@name="server_principal_name"]/value)[1]', 'varchar(100)') AS usuario,
    evento.value('(event/action[@name="sql_text"]/value)[1]', 'varchar(max)') AS instruccion
FROM eventos
ORDER BY fecha_hora DESC;
GO
```

FIGURA 52. DETENCION Y LECTURA DE LOS EVENTOS EXTENDIDOS

	fecha_hora	nombre_evento	usuario	instruccion
1	2026-10-26 23:49:35.5200000	error_reported	DESKTOP-RSGJA07\PC	SELECT folio, estado FROM pedido WHERE estad...
2	2026-10-26 23:49:35.5200000	sql_batch_completed	DESKTOP-RSGJA07\PC	SELECT folio, estado FROM pedido WHERE estad...
3	2026-10-26 23:49:35.5120000	sql_batch_completed	DESKTOP-RSGJA07\PC	USE pepsi_distribucion;

FIGURA 53. RESULTADOS DE LOS EVENTOS UNA VEZ DETENIDOS

CONCLUSIÓN DE LA UNIDAD I

El desarrollo e implementación de la solución integral de administración de bases de datos para el centro de distribución de PepsiCo en Nuevo Laredo, Tamaulipas, consolida una arquitectura informática robusta, altamente disponible y alineada de manera precisa con las necesidades logísticas de la empresa. Al transformar el esquema transaccional de la organización mediante un modelo OLTP adecuadamente dimensionado, el proyecto logra erradicar las inconsistencias operativas derivadas del procesamiento manual de la información, garantizando la consistencia ACID en la captura de pedidos, el control de existencias en tiempo real y el seguimiento detallado de las entregas e incidencias en ruta.

El diseño físico de la base de datos **pepsi_distribucion,** estructurado sobre trece entidades relacionales y respaldado por una segmentación lógica en grupos de archivos independientes (PRIMARY y SECUNDARIO), optimiza el rendimiento de entrada y salida del motor de almacenamiento, mientras que las reglas de integridad del negocio previenen cualquier desviación en el registro de la información. Paralelamente, la postura de seguridad se fortaleció mediante la adopción estricta del principio del mínimo privilegio y el control de acceso basado en roles (RBAC), lo que asegura la adecuada segregación de funciones entre los perfiles de captura, almacén, reparto, supervisión y administración técnica, salvaguardando la confidencialidad e integridad operacional bajo los principios de la norma ISO/IEC 27001 y el modelo CIA.

Finalmente, la integración de un plan de mantenimiento preventivo orientado a la optimización de índices y estadísticas, complementado con mecanismos proactivos de auditoría a través de SQL Server Audit y Eventos Extendidos, garantiza un ciclo de vida

eficiente para el sistema informático, proporcionando trazabilidad total, no repudio y una plataforma escalable preparada para respaldar con solvencia la toma de decisiones estratégicas y la excelencia operativa del centro de distribución.

REFERENCIAS DE LA UNIDAD I

Microsoft Corporation. (2026). Descargas de Microsoft SQL Server y documentación técnica de ediciones. Microsoft. https://www.microsoft.com/es-mx/sql-server/sql-server-downloads

PepsiCo. (s. f.). Portafolio de marcas y productos en México. Pepsi. https://www.pepsi.com.mx/

PepsiCo México. (s. f.). Quienes somos: Misión y visión. PepsiCo. https://www.pepsico.com.mx/quienes-somos/mision-y-vision

Wikipedia. (2026, 12 de febrero). PepsiCo. Wikipedia, La enciclopedia libre. https://es.wikipedia.org/wiki/PepsiCo

CALIDAD DE DATOS Y PYTHON EN EL ALMACÉN

Una base de datos puede tener tablas, permisos y respaldos correctos y aun así recibir información equivocada. En el caso del almacén Pepsi, un precio inválido altera el valor del inventario, un lote mal relacionado puede enviar otra bebida y una fecha incorrecta impide priorizar el producto próximo a caducar. Por eso revisamos los datos antes de utilizarlos para decidir compras y entregas.

Utilizamos datos ficticios para probar el proceso sin utilizar información real de clientes. Faker mantiene una semilla de 7 para repetir el ejercicio con la misma versión de la biblioteca; Pandas permite revisar, filtrar y transformar cada archivo (Faker, s. f.; pandas, s. f.-a, s. f.-b).

En esta versión se ejecutó la generación y la limpieza completa de los tres archivos. Se conservaron los CSV originales, las filas descartadas y los resultados de cada validación. La etapa de conexión y reintegración a SQL Server se entrega en código para ejecutarse con una cuenta autorizada.

TABLAS SELECCIONADAS Y RELACIONES

Tabla	Clave y relaciones	Función
producto	PK id_producto; clave_producto única	Catálogo de bebidas, presentaciones y precios.
lote	PK id_lote; FK id_producto a producto; única (id_producto, numero_lote)	Identificar recepciones y caducidades.
inventario_lote	PK id_inventario; FK (id_lote, id_producto) a lote; FK id_almacen a almacen	Registrar las unidades del lote en el almacén.

Se tomó la estructura de las capturas de creación de tablas del proyecto anterior, específicamente las figuras 6 y 6.1. El almacén 1 sirve como referencia auxiliar; no se

genera un cuarto archivo de análisis. No se agregan columnas de ventas ni subtotales que no existían en esas tablas.

RIESGOS Y CONTROLES DE INTEGRIDAD

Tabla	Riesgo	Control
producto	Un precio negativo o no numérico distorsiona la valuación.	Validar tipo y precio mayor que cero; CHECK en SQL Server.
lote	Una caducidad anterior a la recepción altera la salida de producto.	Comparar fechas; CHECK (fecha_caducidad > fecha_recepcion).
inventario_lote	Relacionar un lote con otro producto o inventar existencias.	Verificar cantidad entera no negativa y relación compuesta; FOREIGN KEY.

Los controles se implementan en las secciones de validación de cada script y, en la versión completa, también mediante las restricciones del código T-SQL. No se convierten precios o cantidades negativos en positivos sin comprobar su valor real; se separan para revisión.

REGLAS DE NEGOCIO POR COLUMNA

Tabla producto

Columna	Regla	Obligatoriedad y relación
id_producto	Entero positivo único	Obligatorio; clave primaria
clave_producto	1 a 15 caracteres; mayúsculas, sin espacios externos	Obligatoria y única
nombre	1 a 100 caracteres; formato título	Obligatorio; bebida comercializada
presentacion	1 a 50 caracteres; minúsculas	Obligatoria
precio	0.01 a 99,999,999.99; máximo dos decimales	Obligatorio; no inventar precios
activo	Entero: 0 o 1	Obligatorio

Tabla lote

Columna	Regla	Obligatoriedad y relación
id_lote	Entero positivo único	Obligatorio; clave primaria
id_producto	Entero positivo existente en producto limpio	Obligatorio; clave foránea
numero_lote	1 a 30 caracteres; mayúsculas	Único junto con id_producto
fecha_recepcion	AAAA-MM-DD; entre 01/01/2026 y 02/11/2026	Obligatoria; fecha de recepción del ejercicio
fecha_caducidad	AAAA-MM-DD; posterior a fecha_recepcion	Obligatoria; consistencia entre columnas

Tabla inventario_lote

Columna	Regla	Obligatoriedad y relación
id_inventario	Entero positivo único	Obligatorio; clave primaria
id_almacen	Entero 1	Almacén de referencia, sin cuarto CSV
id_lote	Entero existente en lote limpio	Único junto con id_almacen
id_producto	Debe coincidir con el producto del lote	Relación compuesta id_lote e id_producto
cantidad	Entero de 0 a 2,147,483,647	Obligatoria; no corregir signos sin evidencia
fecha_actualizacion	AAAA-MM-DD HH:MM:SS; dentro de 2026 y hasta el corte	Obligatoria; corte 02/11/2026 23:59:59

Las claves enteras se limitan al rango de SQL Server INT. No se admiten nulos, texto vacío, espacios externos ni identificadores repetidos. La fecha de corte es el 2 de noviembre de 2026. La presentación se normaliza a minúsculas y las claves de producto y lote a mayúsculas.

DIAGNÓSTICO DE LA CALIDAD DE LOS DATOS

1. ¿Qué porcentaje de registros cumple todas las reglas y permite utilizar la información del almacén? Respuesta: Sí. Después de la limpieza, los registros conservados cumplen las reglas definidas para el ejercicio.

2. ¿Qué datos obligatorios faltan y cómo impedirían identificar el producto o sus existencias? Respuesta: No. En los archivos limpios no faltan datos obligatorios.

3. ¿Existen registros o claves repetidas que puedan inflar el inventario y provocar pedidos innecesarios? Respuesta: No. Los duplicados se identificaron y no permanecen en los archivos limpios.

4. ¿Los precios son numéricos, positivos y válidos para calcular el valor de las existencias? Respuesta: Sí. Los precios aceptados son numéricos y mayores que cero.

5. ¿Todos los lotes e inventarios se relacionan con productos existentes y conservan la relación entre lote y producto? Respuesta: Sí. Se validaron las relaciones entre producto, lote e inventario_lote.

6. ¿Las fechas de recepción y caducidad son válidas y permiten decidir qué lotes deben salir primero? Respuesta: Sí. Las fechas válidas conservan el orden entre recepción y caducidad.

7. ¿Las cantidades son enteras y no negativas, y corresponden al mismo producto y lote para sustentar el reabastecimiento? Respuesta: Sí. Las cantidades aceptadas son enteras no negativas y respetan la relación del lote con el producto.

GENERACIÓN DE LOS TRES ARCHIVOS CSV

Cada archivo contiene 50 filas. Primero se generaron registros relacionados y después se incorporaron fallas intencionales. La última fila se reemplazó por una copia exacta de una fila existente para conservar 50 registros en lugar de aumentar el total a 51.

En producto se incluyeron seis tipos: nulos, duplicados, espacios, mayúsculas o minúsculas inconsistentes, tipos incorrectos y valores inválidos. En lote se incluyeron los nueve tipos de la consigna. En inventario_lote se incluyeron ocho: todos los aplicables, excepto mayúsculas y minúsculas porque sus columnas son numéricas o fechas.

Figura 54

Listado de 01_generar_csv.py, parte 1

```
# Ortega García - Núñez López - Rodríguez Macias
# 01_generar_csv.py | parte 1

  1 # Proyecto Unidad 2
  2 # ? Juan Manuel Ortega García
  3 # ! Bernardo Guillermo Núñez López
  4 # * Carlos Gabriel Rodríguez Macias
  5 # * 2 de noviembre de 2026
  6
  7 # ! Problema
  8 # * Revisar la calidad de los productos, lotes y existencias del almacén
    Pepsi.
  9 # * Los nombres y columnas se tomaron de las figuras 6 y 6.1 del proyecto 1.
 10
 11 # ? Importar las bibliotecas y preparar las carpetas.
 12 import pandas as pd
 13 from faker import Faker
 14 from pathlib import Path
 15 from datetime import date
 16
 17 ruta = Path(__file__).resolve().parent.parent
 18 originales = ruta / 'csv_originales'
 19 originales.mkdir(exist_ok=True)
 20 Faker.seed(7)
 21 fake = Faker('es_MX')
 22
 23 # ? Crear 50 productos de bebidas y presentaciones diferentes.
 24 nombres = ['Pepsi', 'Mirinda', '7 Up', 'Gatorade', 'Mountain Dew']
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 55

Listado de 01_generar_csv.py, parte 2

```
# Ortega García - Núñez López - Rodríguez Macias
# 01_generar_csv.py | parte 2

presentaciones = ['355 ml', '600 ml', '1 litro', '1.5 litros', '2 litros',
                  '3 litros', '6 latas', '12 latas', '24 latas', '12
botellas']
datos = []
for i in range(50):
  datos.append({
    'id_producto': i + 1,
    'clave_producto': f'BEB{i + 1:03}',
    'nombre': nombres[i % 5],
    'presentacion': presentaciones[i // 5],
    'precio': fake.random_int(min=1000, max=30000) / 100,
    'activo': 1
  })
productos = pd.DataFrame(datos)

# ? Agregar nulos, espacios, mayúsculas, tipos incorrectos y valores
inválidos.
# * Un precio perdido o negativo no se recupera inventando otro precio.
productos['precio'] = productos['precio'].astype('object')
productos.loc[0, 'nombre'] = None
productos.loc[1, 'nombre'] = '  Pepsi  '
productos.loc[2, 'presentacion'] = '600 ML'
productos.loc[3, 'precio'] = 'sin precio'
productos.loc[4, 'precio'] = -25
productos.loc[5, 'activo'] = 3
productos.loc[6, 'precio'] = None
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 56

Listado de 01_generar_csv.py, parte 3

```
# Ortega García - Núñez López - Rodríguez Macias
# 01_generar_csv.py | parte 3

 49 productos.loc[7, 'clave_producto'] = '  beb008  '
 50
 51 # ? Reemplazar la última fila por una copia exacta, conservando 50
    registros.
 52 productos.loc[49] = productos.loc[20].copy()
 53 productos.to_csv(originales / 'producto.csv', index=False,
    encoding='utf-8-sig')
 54
 55 # ? Crear 50 lotes relacionados con productos existentes antes de introducir
     fallas.
 56 datos = []
 57 for i in range(50):
 58   recepcion = fake.date_between(date(2026, 1, 1), date(2026, 9, 30))
 59   datos.append({
 60     'id_lote': i + 1,
 61     'id_producto': (i % 49) + 1,
 62     'numero_lote': f'LOT{i + 1:03}',
 63     'fecha_recepcion': recepcion.isoformat(),
 64     'fecha_caducidad': date(2027, recepcion.month,
    recepcion.day).isoformat()
 65   })
 66 lotes = pd.DataFrame(datos)
 67 lotes['id_producto'] = lotes['id_producto'].astype('object')
 68
 69 # ? Introducir los nueve tipos de error que aplican a los lotes.
 70 lotes.loc[0, 'numero_lote'] = None
 71 lotes.loc[1, 'numero_lote'] = '  LOT002  '
 72 lotes.loc[2, 'numero_lote'] = 'lot003'
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 57

Listado de 01_generar_csv.py, parte 4

```
# Ortega García - Núñez López - Rodríguez Macias
# 01_generar_csv.py | parte 4

lotes.loc[3, 'fecha_recepcion'] = '15/03/2026'
lotes.loc[4, 'id_producto'] = 'producto cinco'
lotes.loc[5, 'id_producto'] = 999
lotes.loc[6, 'fecha_caducidad'] = '2025-01-01'
lotes.loc[7, 'fecha_recepcion'] = '2026-02-30'
lotes.loc[8, 'fecha_recepcion'] = '2025-12-01'
lotes.loc[49] = lotes.loc[20].copy()
lotes.to_csv(originales / 'lote.csv', index=False, encoding='utf-8-sig')

# ? Crear 50 registros de inventario para el almacén 1.
# * Se conserva la relación compuesta id_lote e id_producto de la captura original.
datos = []
for i in range(50):
  datos.append({
    'id_inventario': i + 1,
    'id_almacen': 1,
    'id_lote': i + 1,
    'id_producto': (i % 49) + 1,
    'cantidad': fake.random_int(min=0, max=500),
    'fecha_actualizacion': '2026-10-31 12:00:00'
  })
inventario = pd.DataFrame(datos)
inventario['cantidad'] = inventario['cantidad'].astype('object')

```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 58

Listado de 01_generar_csv.py, parte 5

```
# Ortega García - Núñez López - Rodríguez Macias
# 01_generar_csv.py | parte 5

 97 # ? Agregar errores de cantidad, fechas y relaciones sin inventar columnas
    nuevas.
 98 # * No se prueba mayúsculas en columnas numéricas porque no corresponde.
 99 inventario.loc[0, 'cantidad'] = None
100 inventario.loc[1, 'fecha_actualizacion'] = ' 2026-10-31 12:00:00 '
101 inventario.loc[2, 'fecha_actualizacion'] = '31/10/2026 12:00:00'
102 inventario.loc[3, 'cantidad'] = 'muchas'
103 inventario.loc[4, 'cantidad'] = -12
104 inventario.loc[5, 'id_lote'] = 999
105 inventario.loc[6, 'id_producto'] = 40
106 inventario.loc[7, 'fecha_actualizacion'] = '2026-02-30 12:00:00'
107 inventario.loc[8, 'cantidad'] = 1.5
108 inventario.loc[49] = inventario.loc[20].copy()
109 inventario.to_csv(originales / 'inventario_lote.csv', index=False,
    encoding='utf-8-sig')
110
111 # ? Comprobar que los tres CSV contienen exactamente 50 filas.
112 print('Ortega García - Núñez López - Rodríguez Macias')
113 print('producto:', len(productos), 'registros')
114 print('lote:', len(lotes), 'registros')
115 print('inventario_lote:', len(inventario), 'registros')
116
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Seleccionamos estos errores porque pueden surgir al copiar catálogos, repetir una carga, omitir campos, escribir un precio con texto o capturar una fecha con otra configuración regional. Una relación rota puede aparecer cuando el archivo de existencias llega antes que su catálogo. La discrepancia entre lote y producto simula seleccionar una bebida distinta durante la captura del almacén.

Error	Por qué se prueba	Ejemplo en producción
Valor nulo	Comprobar campos obligatorios.	Se omite el nombre del producto o su cantidad al capturar.
Duplicado exacto	Evitar contar dos veces la misma información.	Se vuelve a cargar el archivo de una recepción.
Espacios externos	Evitar diferencias falsas al comparar textos.	Se copia una clave desde una hoja de cálculo.

Error	Por qué se prueba	Ejemplo en producción
Mayúsculas y minúsculas	Homologar claves y presentaciones.	Dos operadores escriben LOT003 y lot003.
Formato de fecha	Interpretar correctamente recepción y caducidad.	Una estación exporta día/mes/año y otra año-mes-día.
Tipo incorrecto	Detectar texto donde se requiere un número.	Se escribe muchas en la cantidad o sin precio en el catálogo.
Valor inválido	Impedir precios negativos, fechas imposibles o cantidades fraccionarias.	Un error de signo registra -12 unidades disponibles.
Relación rota	Mantener la trazabilidad entre archivos.	Llega una existencia que apunta a un lote no registrado.
Inconsistencia entre columnas	Comprobar que los campos sean compatibles.	El lote pertenece a otra bebida o caduca antes de su recepción.

LECTURA Y BÚSQUEDA DE ERRORES

Cada tabla cuenta con su propio archivo Python. Se utiliza try-except para avisar si falta el CSV y no continuar con datos inexistentes. info() muestra columnas y tipos; shape muestra filas y columnas; head() permite inspeccionar una muestra. La versión completa también emplea describe(include="all") para revisar valores frecuentes y distribución.

Los errores se buscan por reglas sobre todas las filas; las posiciones usadas por el generador no se utilizan para decidir cuáles registros limpiar. Los valores que no se pueden convertir se detectan con errores controlados en lugar de detener el análisis (pandas, s. f.-a, s. f.-b).

Figura 59

Listado de 02_revisar_producto.py, parte 1

```
# Ortega García - Núñez López - Rodríguez Macias
# 02_revisar_producto.py | parte 1

  1 # Proyecto Unidad 2
  2 # ? Juan Manuel Ortega García
  3 # ! Bernardo Guillermo Núñez López
  4 # * Carlos Gabriel Rodríguez Macias
  5 # * 2 de noviembre de 2026
  6
  7 # ? Importar el CSV y controlar la ausencia del archivo o de sus
    referencias.
  8 import pandas as pd
  9 from pathlib import Path
 10
 11 ruta = Path(__file__).resolve().parent.parent
 12 try:
 13   datos_csv = pd.read_csv(ruta / 'csv_originales' / 'producto.csv',
    dtype='string')
 14 except (FileNotFoundError, pd.errors.EmptyDataError, pd.errors.ParserError)
    as error:
 15   print('No se pudo leer el archivo:', error)
 16   raise SystemExit(1)
 17
 18 # ? Mostrar la estructura, dimensiones y estadísticas antes de limpiar.
 19 print('Ortega García - Núñez López - Rodríguez Macias')
 20 datos_csv.info()
 21 print('Dimensiones:', datos_csv.shape)
 22 print(datos_csv.describe(include='all'))
 23
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 60

Listado de 02_revisar_producto.py, parte 2

```
# Ortega García - Núñez López - Rodríguez Macias
# 02_revisar_producto.py | parte 2

 24 # ? Buscar errores por reglas generales, sin usar las posiciones de las
    fallas.
 25 # * Esta función se ejecuta antes y después con las mismas reglas.
 26 def validar(datos):
 27   errores = pd.DataFrame(index=datos.index)
 28   for columna in datos.columns:
 29     errores[columna + '_nulo'] = datos[columna].isna() |
    datos[columna].str.strip().eq('').fillna(False)
 30     errores[columna + '_espacios'] =
    datos[columna].ne(datos[columna].str.strip()).fillna(False)
 31   errores['duplicado_exacto'] = datos.duplicated(keep=False)
 32   errores['clave_primaria_repetida'] =
    datos.duplicated(subset=['id_producto'], keep=False)
 33   numeros = {}
 34   fechas = {}
 35   numeros['id_producto'] = pd.to_numeric(datos['id_producto'],
    errors='coerce')
 36   errores['id_producto_tipo'] = numeros['id_producto'].isna() |
    numeros['id_producto'].isin([float('inf'), -float('inf')])
 37   numeros['precio'] = pd.to_numeric(datos['precio'], errors='coerce')
 38   errores['precio_tipo'] = numeros['precio'].isna() |
    numeros['precio'].isin([float('inf'), -float('inf')])
 39   numeros['activo'] = pd.to_numeric(datos['activo'], errors='coerce')
 40   errores['activo_tipo'] = numeros['activo'].isna() |
    numeros['activo'].isin([float('inf'), -float('inf')])
 41   errores['clave_producto_formato'] = datos['clave_producto'].ne(datos['clav
    e_producto'].str.strip().str.upper()).fillna(False)
 42   errores['nombre_formato'] =
    datos['nombre'].ne(datos['nombre'].str.strip().str.title()).fillna(False)
 43   errores['presentacion_formato'] = datos['presentacion'].ne(datos['presenta
    cion'].str.strip().str.lower()).fillna(False)
 44   errores['clave_producto_longitud'] =
    datos['clave_producto'].str.len().gt(15).fillna(False)
 45   errores['nombre_longitud'] =
    datos['nombre'].str.len().gt(100).fillna(False)
 46   errores['presentacion_longitud'] =
    datos['presentacion'].str.len().gt(50).fillna(False)
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 61

Listado de 02_revisar_producto.py, parte 3

```
# Ortega García - Núñez López - Rodríguez Macias
# 02_revisar_producto.py | parte 3

 47   errores['clave_negocio_1_repetida'] =
    datos.duplicated(subset=['clave_producto'], keep=False)
 48
 49   errores['id_producto_invalido'] = ~numeros['id_producto'].between(1,
    2147483647) | numeros['id_producto'].mod(1).ne(0)
 50   errores['precio_invalido'] = ~numeros['precio'].between(0.01, 99999999.99)
 51   errores['precio_decimales'] =
    numeros['precio'].sub(numeros['precio'].round(2)).abs().gt(0.000001)
 52   errores['activo_invalido'] = ~numeros['activo'].isin([0, 1])
 53   return errores.fillna(False)
 54
 55 # ? Contar filas aptas una sola vez aunque tengan varios errores.
 56 antes = validar(datos_csv)
 57 no_aptos = antes.any(axis=1)
 58 print('Errores por regla:')
 59 print(antes.sum())
 60 print('Filas afectadas:')
 61 print(datos_csv.loc[no_aptos].to_string(index=False))
 62 original = datos_csv.copy()
 63
 64 # ? Normalizar texto y conservar el original para comparar los cambios.
 65 limpios = datos_csv.copy()
 66 for columna in limpios.columns:
 67   limpios[columna] = limpios[columna].str.strip()
 68 limpios['clave_producto'] = limpios['clave_producto'].str.upper()
 69 limpios['nombre'] = limpios['nombre'].str.title()
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 62

Listado de 02_revisar_producto.py, parte 4

```
# Ortega García - Núñez López - Rodríguez Macias
# 02_revisar_producto.py | parte 4

 70 limpios['presentacion'] = limpios['presentacion'].str.lower()
 71
 72
 73 # ? Retirar únicamente las repeticiones exactas y separar filas sin solución
     comprobable.
 74 limpios = limpios.drop_duplicates(keep='first').copy()
 75 pendientes = validar(limpios).any(axis=1)
 76 limpios = limpios.loc[~pendientes].copy()
 77 despues = validar(limpios)
 78 if despues.any(axis=1).any():
 79   raise ValueError('Todavía existen errores; no se exportará como archivo
    limpio.')
 80
 81 # ? Registrar conservados, corregidos y descartados sin contar una fila dos
    veces.
 82 conservados = len(limpios)
 83 descartados = len(original) - conservados
 84 modificados =
    original.loc[limpios.index].fillna('').ne(limpios.fillna('')).any(axis=1)
 85 corregidos = int(modificados.sum())
 86 sin_cambios = conservados - corregidos
 87 resumen = pd.DataFrame([{
 88   'archivo': 'producto', 'iniciales': len(original),
 89   'aptos_antes': int((~no_aptos).sum()), 'no_aptos_antes':
    int(no_aptos.sum()),
 90   'porcentaje_apto_antes': round((~no_aptos).mean() * 100, 2),
 91   'porcentaje_no_apto_antes': round(no_aptos.mean() * 100, 2),
 92   'conservados': conservados, 'corregidos': corregidos,
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 63

Listado de 02_revisar_producto.py, parte 5

```
# Ortega García - Núñez López - Rodríguez Macias
# 02_revisar_producto.py | parte 5

 93   'sin_cambios': sin_cambios, 'descartados': descartados,
 94   'aptos_despues': conservados, 'no_aptos_despues': 0,
 95   'porcentaje_apto_despues': 100.0 if conservados else 0.0,
 96   'porcentaje_no_apto_despues': 0.0
 97 }])
 98 print('Resumen de calidad:')
 99 for columna, valor in resumen.iloc[0].items():
100   print(f'{columna}: {valor}')
101
102 # ? Guardar los hallazgos por fila y los archivos listos para SQL Server.
103 hallazgos = antes.copy()
104 hallazgos.insert(0, 'fila_csv', original.index + 2)
105 hallazgos.to_csv(ruta / 'resultados' / 'producto_errores.csv', index=False)
106 descartes = original.loc[~original.index.isin(limpios.index)].copy()
107 descartes.insert(0, 'fila_csv', descartes.index + 2)
108 descartes.to_csv(ruta / 'resultados' / 'producto_descartados.csv',
    index=False)
109 resumen.to_csv(ruta / 'resultados' / 'producto_resumen.csv', index=False)
110 for columna in ['id_producto','activo']:
111   limpios[columna] = pd.to_numeric(limpios[columna]).astype('int64')
112 limpios['precio'] = pd.to_numeric(limpios['precio']).round(2)
113 limpios.to_csv(ruta / 'csv_limpios' / 'producto.csv', index=False,
    encoding='utf-8-sig')
114 print('Validación final: 0 filas no aptas.')
115
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 64

Listado de 03_revisar_lote.py, parte 1

```
# Ortega García - Núñez López - Rodríguez Macias
# 03_revisar_lote.py | parte 1

  1 # Proyecto Unidad 2
  2 # ? Juan Manuel Ortega García
  3 # ! Bernardo Guillermo Núñez López
  4 # * Carlos Gabriel Rodríguez Macias
  5 # * 2 de noviembre de 2026
  6
  7 # ? Importar el CSV y controlar la ausencia del archivo o de sus
    referencias.
  8 import pandas as pd
  9 from pathlib import Path
 10
 11 ruta = Path(__file__).resolve().parent.parent
 12 try:
 13   datos_csv = pd.read_csv(ruta / 'csv_originales' / 'lote.csv',
    dtype='string')
 14   productos = pd.read_csv(ruta / 'csv_limpios' / 'producto.csv')
 15 except (FileNotFoundError, pd.errors.EmptyDataError, pd.errors.ParserError)
    as error:
 16   print('No se pudo leer el archivo:', error)
 17   raise SystemExit(1)
 18
 19 # ? Mostrar la estructura, dimensiones y estadísticas antes de limpiar.
 20 print('Ortega García - Núñez López - Rodríguez Macias')
 21 datos_csv.info()
 22 print('Dimensiones:', datos_csv.shape)
 23 print(datos_csv.describe(include='all'))
 24
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 65

Listado de 03_revisar_lote.py, parte 2

```
# Ortega García - Núñez López - Rodríguez Macias
# 03_revisar_lote.py | parte 2

# ? Buscar errores por reglas generales, sin usar las posiciones de las fallas.
# * Esta función se ejecuta antes y después con las mismas reglas.
def validar(datos):
  errores = pd.DataFrame(index=datos.index)
  for columna in datos.columns:
    errores[columna + '_nulo'] = datos[columna].isna() | datos[columna].str.strip().eq('').fillna(False)
    errores[columna + '_espacios'] = datos[columna].ne(datos[columna].str.strip()).fillna(False)
  errores['duplicado_exacto'] = datos.duplicated(keep=False)
  errores['clave_primaria_repetida'] = datos.duplicated(subset=['id_lote'], keep=False)
  numeros = {}
  fechas = {}
  numeros['id_lote'] = pd.to_numeric(datos['id_lote'], errors='coerce')
  errores['id_lote_tipo'] = numeros['id_lote'].isna() | numeros['id_lote'].isin([float('inf'), -float('inf')])
  numeros['id_producto'] = pd.to_numeric(datos['id_producto'], errors='coerce')
  errores['id_producto_tipo'] = numeros['id_producto'].isna() | numeros['id_producto'].isin([float('inf'), -float('inf')])
  errores['numero_lote_formato'] = datos['numero_lote'].ne(datos['numero_lote'].str.strip().str.upper()).fillna(False)
  errores['numero_lote_longitud'] = datos['numero_lote'].str.len().gt(30).fillna(False)
  errores['clave_negocio_1_repetida'] = datos.duplicated(subset=['id_producto','numero_lote'], keep=False)
  fechas['fecha_recepcion'] = pd.to_datetime(datos['fecha_recepcion'], format='%Y-%m-%d', errors='coerce')
  errores['fecha_recepcion_invalida'] = fechas['fecha_recepcion'].isna()
  errores['fecha_recepcion_formato'] = datos['fecha_recepcion'].ne(fechas['fecha_recepcion'].dt.strftime('%Y-%m-%d')).fillna(True)
  fechas['fecha_caducidad'] = pd.to_datetime(datos['fecha_caducidad'], format='%Y-%m-%d', errors='coerce')
  errores['fecha_caducidad_invalida'] = fechas['fecha_caducidad'].isna()
  errores['fecha_caducidad_formato'] = datos['fecha_caducidad'].ne(fechas['fecha_caducidad'].dt.strftime('%Y-%m-%d')).fillna(True)
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 66

Listado de 03_revisar_lote.py, parte 3

```
# Ortega García - Núñez López - Rodríguez Macias
# 03_revisar_lote.py | parte 3

 49   for columna in ['id_lote', 'id_producto']:
 50     errores[columna + '_invalido'] = ~numeros[columna].between(1,
    2147483647) | numeros[columna].mod(1).ne(0)
 51   errores['producto_inexistente'] =
    ~numeros['id_producto'].isin(productos['id_producto'])
 52   errores['recepcion_fuera_periodo'] =
    ~fechas['fecha_recepcion'].between('2026-01-01', '2026-11-02')
 53   errores['caducidad_antes_recepcion'] =
    fechas['fecha_caducidad'].le(fechas['fecha_recepcion'])
 54   return errores.fillna(False)
 55
 56 # ? Contar filas aptas una sola vez aunque tengan varios errores.
 57 antes = validar(datos_csv)
 58 no_aptos = antes.any(axis=1)
 59 print('Errores por regla:')
 60 print(antes.sum())
 61 print('Filas afectadas:')
 62 print(datos_csv.loc[no_aptos].to_string(index=False))
 63 original = datos_csv.copy()
 64
 65 # ? Normalizar texto y conservar el original para comparar los cambios.
 66 limpios = datos_csv.copy()
 67 for columna in limpios.columns:
 68   limpios[columna] = limpios[columna].str.strip()
 69 limpios['numero_lote'] = limpios['numero_lote'].str.upper()
 70 # * Aceptar dos formatos conocidos sin adivinar el orden de día y mes.
 71 fechas = pd.to_datetime(limpios['fecha_recepcion'], format='%Y-%m-%d',
    errors='coerce')
 72 alternativas = pd.to_datetime(limpios['fecha_recepcion'], format='%d/%m/%Y',
     errors='coerce')
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 67

Listado de 03_revisar_lote.py, parte 4

```
# Ortega García - Núñez López - Rodríguez Macias
# 03_revisar_lote.py | parte 4

 73 limpios['fecha_recepcion'] =
    fechas.fillna(alternativas).dt.strftime('%Y-%m-%d').astype('string')
 74 # * Aceptar dos formatos conocidos sin adivinar el orden de día y mes.
 75 fechas = pd.to_datetime(limpios['fecha_caducidad'], format='%Y-%m-%d',
    errors='coerce')
 76 alternativas = pd.to_datetime(limpios['fecha_caducidad'], format='%d/%m/%Y',
     errors='coerce')
 77 limpios['fecha_caducidad'] =
    fechas.fillna(alternativas).dt.strftime('%Y-%m-%d').astype('string')
 78
 79 # ? Retirar únicamente las repeticiones exactas y separar filas sin solución
     comprobable.
 80 limpios = limpios.drop_duplicates(keep='first').copy()
 81 pendientes = validar(limpios).any(axis=1)
 82 limpios = limpios.loc[~pendientes].copy()
 83 despues = validar(limpios)
 84 if despues.any(axis=1).any():
 85   raise ValueError('Todavía existen errores; no se exportará como archivo
    limpio.')
 86
 87 # ? Registrar conservados, corregidos y descartados sin contar una fila dos
    veces.
 88 conservados = len(limpios)
 89 descartados = len(original) - conservados
 90 modificados =
    original.loc[limpios.index].fillna('').ne(limpios.fillna('')).any(axis=1)
 91 corregidos = int(modificados.sum())
 92 sin_cambios = conservados - corregidos
 93 resumen = pd.DataFrame([{
 94   'archivo': 'lote', 'iniciales': len(original),
 95   'aptos_antes': int((~no_aptos).sum()), 'no_aptos_antes':
    int(no_aptos.sum()),
 96   'porcentaje_apto_antes': round((~no_aptos).mean() * 100, 2),
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 68

Listado de 03_revisar_lote.py, parte 5

```
# Ortega García - Núñez López - Rodríguez Macias
# 03_revisar_lote.py | parte 5

 97   'porcentaje_no_apto_antes': round(no_aptos.mean() * 100, 2),
 98   'conservados': conservados, 'corregidos': corregidos,
 99   'sin_cambios': sin_cambios, 'descartados': descartados,
100   'aptos_despues': conservados, 'no_aptos_despues': 0,
101   'porcentaje_apto_despues': 100.0 if conservados else 0.0,
102   'porcentaje_no_apto_despues': 0.0
103 }])
104 print('Resumen de calidad:')
105 for columna, valor in resumen.iloc[0].items():
106   print(f'{columna}: {valor}')
107
108 # ? Guardar los hallazgos por fila y los archivos listos para SQL Server.
109 hallazgos = antes.copy()
110 hallazgos.insert(0, 'fila_csv', original.index + 2)
111 hallazgos.to_csv(ruta / 'resultados' / 'lote_errores.csv', index=False)
112 descartes = original.loc[~original.index.isin(limpios.index)].copy()
113 descartes.insert(0, 'fila_csv', descartes.index + 2)
114 descartes.to_csv(ruta / 'resultados' / 'lote_descartados.csv', index=False)
115 resumen.to_csv(ruta / 'resultados' / 'lote_resumen.csv', index=False)
116 for columna in ['id_lote','id_producto']:
117   limpios[columna] = pd.to_numeric(limpios[columna]).astype('int64')
118 limpios.to_csv(ruta / 'csv_limpios' / 'lote.csv', index=False,
    encoding='utf-8-sig')
119 print('Validación final: 0 filas no aptas.')
120
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 69

Listado de 04_revisar_inventario_lote.py, parte 1

```
# Ortega García - Núñez López - Rodríguez Macias
# 04_revisar_inventario_lote.py | parte 1

  1 # Proyecto Unidad 2
  2 # ? Juan Manuel Ortega García
  3 # ! Bernardo Guillermo Núñez López
  4 # * Carlos Gabriel Rodríguez Macias
  5 # * 2 de noviembre de 2026
  6
  7 # ? Importar el CSV y controlar la ausencia del archivo o de sus
    referencias.
  8 import pandas as pd
  9 from pathlib import Path
 10
 11 ruta = Path(__file__).resolve().parent.parent
 12 try:
 13   datos_csv = pd.read_csv(ruta / 'csv_originales' / 'inventario_lote.csv',
    dtype='string')
 14   productos = pd.read_csv(ruta / 'csv_limpios' / 'producto.csv')
 15   lotes = pd.read_csv(ruta / 'csv_limpios' / 'lote.csv')
 16 except (FileNotFoundError, pd.errors.EmptyDataError, pd.errors.ParserError)
    as error:
 17   print('No se pudo leer el archivo:', error)
 18   raise SystemExit(1)
 19
 20 # ? Mostrar la estructura, dimensiones y estadísticas antes de limpiar.
 21 print('Ortega García - Núñez López - Rodríguez Macias')
 22 datos_csv.info()
 23 print('Dimensiones:', datos_csv.shape)
 24 print(datos_csv.describe(include='all'))
 25
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 70

Listado de 04_revisar_inventario_lote.py, parte 2

```
# Ortega García - Núñez López - Rodríguez Macias
# 04_revisar_inventario_lote.py | parte 2

# ? Buscar errores por reglas generales, sin usar las posiciones de las fallas.
# * Esta función se ejecuta antes y después con las mismas reglas.
def validar(datos):
  errores = pd.DataFrame(index=datos.index)
  for columna in datos.columns:
    errores[columna + '_nulo'] = datos[columna].isna() | datos[columna].str.strip().eq('').fillna(False)
    errores[columna + '_espacios'] = datos[columna].ne(datos[columna].str.strip()).fillna(False)
  errores['duplicado_exacto'] = datos.duplicated(keep=False)
  errores['clave_primaria_repetida'] = datos.duplicated(subset=['id_inventario'], keep=False)
  numeros = {}
  fechas = {}
  numeros['id_inventario'] = pd.to_numeric(datos['id_inventario'], errors='coerce')
  errores['id_inventario_tipo'] = numeros['id_inventario'].isna() | numeros['id_inventario'].isin([float('inf'), -float('inf')])
  numeros['id_almacen'] = pd.to_numeric(datos['id_almacen'], errors='coerce')
  errores['id_almacen_tipo'] = numeros['id_almacen'].isna() | numeros['id_almacen'].isin([float('inf'), -float('inf')])
  numeros['id_lote'] = pd.to_numeric(datos['id_lote'], errors='coerce')
  errores['id_lote_tipo'] = numeros['id_lote'].isna() | numeros['id_lote'].isin([float('inf'), -float('inf')])
  numeros['id_producto'] = pd.to_numeric(datos['id_producto'], errors='coerce')
  errores['id_producto_tipo'] = numeros['id_producto'].isna() | numeros['id_producto'].isin([float('inf'), -float('inf')])
  numeros['cantidad'] = pd.to_numeric(datos['cantidad'], errors='coerce')
  errores['cantidad_tipo'] = numeros['cantidad'].isna() | numeros['cantidad'].isin([float('inf'), -float('inf')])


  errores['clave_negocio_1_repetida'] = datos.duplicated(subset=['id_almacen','id_lote'], keep=False)
  fechas['fecha_actualizacion'] = pd.to_datetime(datos['fecha_actualizacion'], format='%Y-%m-%d %H:%M:%S', errors='coerce')
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 71

Listado de 04_revisar_inventario_lote.py, parte 3

```
# Ortega García - Núñez López - Rodríguez Macias
# 04_revisar_inventario_lote.py | parte 3

 51   errores['fecha_actualizacion_invalida'] =
    fechas['fecha_actualizacion'].isna()
 52   errores['fecha_actualizacion_formato'] = datos['fecha_actualizacion'].ne(f
    echas['fecha_actualizacion'].dt.strftime('%Y-%m-%d %H:%M:%S')).fillna(True)
 53   for columna in ['id_inventario', 'id_almacen', 'id_lote', 'id_producto']:
 54     errores[columna + '_invalido'] = ~numeros[columna].between(1,
    2147483647) | numeros[columna].mod(1).ne(0)
 55   errores['cantidad_invalida'] = ~numeros['cantidad'].between(0, 2147483647)
     | numeros['cantidad'].mod(1).ne(0)
 56   errores['almacen_inexistente'] = ~numeros['id_almacen'].isin([1])
 57   errores['producto_inexistente'] =
    ~numeros['id_producto'].isin(productos['id_producto'])
 58   errores['lote_inexistente'] = ~numeros['id_lote'].isin(lotes['id_lote'])
 59   producto_por_lote = lotes.set_index('id_lote')['id_producto']
 60   errores['producto_no_corresponde_lote'] =
    numeros['id_producto'].ne(numeros['id_lote'].map(producto_por_lote))
 61   errores['actualizacion_fuera_periodo'] =
    ~fechas['fecha_actualizacion'].between('2026-01-01', '2026-11-02 23:59:59')
 62   return errores.fillna(False)
 63
 64 # ? Contar filas aptas una sola vez aunque tengan varios errores.
 65 antes = validar(datos_csv)
 66 no_aptos = antes.any(axis=1)
 67 print('Errores por regla:')
 68 print(antes.sum())
 69 print('Filas afectadas:')
 70 print(datos_csv.loc[no_aptos].to_string(index=False))
 71 original = datos_csv.copy()
 72
 73 # ? Normalizar texto y conservar el original para comparar los cambios.
 74 limpios = datos_csv.copy()
 75 for columna in limpios.columns:
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 72

Listado de 04_revisar_inventario_lote.py, parte 4

```
# Ortega García - Núñez López - Rodríguez Macias
# 04_revisar_inventario_lote.py | parte 4

 76   limpios[columna] = limpios[columna].str.strip()
 77
 78 # * Aceptar dos formatos conocidos sin adivinar el orden de día y mes.
 79 fechas = pd.to_datetime(limpios['fecha_actualizacion'], format='%Y-%m-%d
    %H:%M:%S', errors='coerce')
 80 alternativas = pd.to_datetime(limpios['fecha_actualizacion'],
    format='%d/%m/%Y %H:%M:%S', errors='coerce')
 81 limpios['fecha_actualizacion'] =
    fechas.fillna(alternativas).dt.strftime('%Y-%m-%d
    %H:%M:%S').astype('string')
 82
 83 # ? Retirar únicamente las repeticiones exactas y separar filas sin solución
     comprobable.
 84 limpios = limpios.drop_duplicates(keep='first').copy()
 85 pendientes = validar(limpios).any(axis=1)
 86 limpios = limpios.loc[~pendientes].copy()
 87 despues = validar(limpios)
 88 if despues.any(axis=1).any():
 89   raise ValueError('Todavía existen errores; no se exportará como archivo
    limpio.')
 90
 91 # ? Registrar conservados, corregidos y descartados sin contar una fila dos
    veces.
 92 conservados = len(limpios)
 93 descartados = len(original) - conservados
 94 modificados =
    original.loc[limpios.index].fillna('').ne(limpios.fillna('')).any(axis=1)
 95 corregidos = int(modificados.sum())
 96 sin_cambios = conservados - corregidos
 97 resumen = pd.DataFrame([{
 98   'archivo': 'inventario_lote', 'iniciales': len(original),
 99   'aptos_antes': int((~no_aptos).sum()), 'no_aptos_antes':
    int(no_aptos.sum()),
100   'porcentaje_apto_antes': round((~no_aptos).mean() * 100, 2),
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 73

Listado de 04_revisar_inventario_lote.py, parte 5

```
# Ortega García - Núñez López - Rodríguez Macias
# 04_revisar_inventario_lote.py | parte 5

  'porcentaje_no_apto_antes': round(no_aptos.mean() * 100, 2),
  'conservados': conservados, 'corregidos': corregidos,
  'sin_cambios': sin_cambios, 'descartados': descartados,
  'aptos_despues': conservados, 'no_aptos_despues': 0,
  'porcentaje_apto_despues': 100.0 if conservados else 0.0,
  'porcentaje_no_apto_despues': 0.0
}])
print('Resumen de calidad:')
for columna, valor in resumen.iloc[0].items():
  print(f'{columna}: {valor}')

# ? Guardar los hallazgos por fila y los archivos listos para SQL Server.
hallazgos = antes.copy()
hallazgos.insert(0, 'fila_csv', original.index + 2)
hallazgos.to_csv(ruta / 'resultados' / 'inventario_lote_errores.csv',
    index=False)
descartes = original.loc[~original.index.isin(limpios.index)].copy()
descartes.insert(0, 'fila_csv', descartes.index + 2)
descartes.to_csv(ruta / 'resultados' / 'inventario_lote_descartados.csv',
    index=False)
resumen.to_csv(ruta / 'resultados' / 'inventario_lote_resumen.csv',
    index=False)
for columna in
    ['id_inventario','id_almacen','id_lote','id_producto','cantidad']:
  limpios[columna] = pd.to_numeric(limpios[columna]).astype('int64')
limpios.to_csv(ruta / 'csv_limpios' / 'inventario_lote.csv', index=False,
    encoding='utf-8-sig')
print('Validación final: 0 filas no aptas.')

```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

LIMPIEZA Y TRANSFORMACIÓN MEDIANTE ETL

ETL consiste en extraer los datos, transformarlos y cargarlos en un destino. En este proyecto la extracción lee los CSV originales, la transformación detecta y corrige inconsistencias comprobables y la carga reintegra los archivos aptos en SQL Server. Este orden evita trasladar fallas del archivo de origen al sistema operativo del almacén (Microsoft, s. f.-b).

Se recortaron espacios, se uniformaron textos y se convirtieron dos formatos de fecha conocidos. Se conservó una sola copia de cada duplicado exacto. Los registros con nulos obligatorios, importes no recuperables, cantidades inválidas o relaciones rotas se descartaron del archivo apto, pero permanecen en los archivos de revisión. Los lotes e inventarios se compararon contra sus referencias ya limpias.

Una fila puede incumplir varias reglas. Para calcular aptos y no aptos se utiliza una condición por fila, de manera que no se suman los errores como si fueran registros diferentes. Después de limpiar se ejecuta la misma función de validación.

Archivo y error	Columna	Filas CSV	Operación	Consecuencia
producto: clave_producto_espacios	clave_producto	9	Normalizar texto	Conteo o valor del inventario incorrecto
producto: nombre_nulo	nombre	2	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto
producto: nombre_espacios	nombre	3	Normalizar texto	Conteo o valor del inventario incorrecto
producto: precio_nulo	precio	8	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto

Archivo y error	Columna	Filas CSV	Operación	Consecuencia
producto: duplicado_exacto	Todas	22, 51	Conservar una copia; revisar claves	Conteo o valor del inventario incorrecto
producto: clave_primaria_repetida	id_producto	22, 51	Conservar una copia; revisar claves	Conteo o valor del inventario incorrecto
producto: precio_tipo	precio	5, 8	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto
producto: clave_producto_formato	clave_producto	9	Normalizar texto	Conteo o valor del inventario incorrecto
producto: nombre_formato	nombre	3	Normalizar texto	Conteo o valor del inventario incorrecto
producto: presentacion_formato	presentacion	4	Normalizar texto	Conteo o valor del inventario incorrecto
producto: clave_negocio_1_repetida	clave_producto	22, 51	Conservar una copia; revisar claves	Conteo o valor del inventario incorrecto
producto: precio_invalido	precio	6	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto
producto: activo_invalido	activo	7	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto
lote: numero_lote_nulo	numero_lote	2	Separar registro si no es recuperable	Trazabilidad o caducidad incorrecta
lote: numero_lote_espacios	numero_lote	3	Normalizar texto	Trazabilidad o caducidad incorrecta
lote: duplicado_exacto	Todas	22, 51	Conservar una copia; revisar claves	Trazabilidad o caducidad incorrecta

Archivo y error	Columna	Filas CSV	Operación	Consecuencia
lote: clave_primaria_repetida	id_lote	22, 51	Conservar una copia; revisar claves	Trazabilidad o caducidad incorrecta
lote: id_producto_tipo	id_producto	6	Separar registro si no es recuperable	Trazabilidad o caducidad incorrecta
lote: numero_lote_formato	numero_lote	3, 4	Normalizar texto	Trazabilidad o caducidad incorrecta
lote: clave_negocio_1_repetida	id_producto / numero_lote	22, 51	Conservar una copia; revisar claves	Trazabilidad o caducidad incorrecta
lote: fecha_recepcion_invalida	fecha_recepcion	5, 9	Separar registro si no es recuperable	Trazabilidad o caducidad incorrecta
lote: fecha_recepcion_formato	fecha_recepcion	5, 9	Normalizar texto	Trazabilidad o caducidad incorrecta
lote: producto_inexistente	id_producto	2, 5, 6, 7, 8	Separar registro si no es recuperable	Trazabilidad o caducidad incorrecta
lote: recepcion_fuera_periodo	fecha_recepcion	5, 9, 10	Separar registro si no es recuperable	Trazabilidad o caducidad incorrecta
lote: caducidad_antes_recepcion	fecha_recepcion / fecha_caducidad	8	Separar registro si no es recuperable	Trazabilidad o caducidad incorrecta
inventario_lote: cantidad_nulo	cantidad	2	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto
inventario_lote: fecha_actualizacion_espacios	fecha_actualizacion	3	Normalizar texto	Conteo o valor del inventario incorrecto
inventario_lote: duplicado_exacto	Todas	22, 51	Conservar una copia; revisar claves	Conteo o valor del inventario incorrecto

Archivo y error	Columna	Filas CSV	Operación	Consecuencia
inventario_lote: clave_primaria_repetida	id_inventario	22, 51	Conservar una copia; revisar claves	Conteo o valor del inventario incorrecto
inventario_lote: cantidad_tipo	cantidad	2, 5	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto
inventario_lote: clave_negocio_1_repetida	id_almacen / id_lote	22, 51	Conservar una copia; revisar claves	Conteo o valor del inventario incorrecto
inventario_lote: fecha_actualizacion_invalida	fecha_actualizacion	3, 4, 9	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto
inventario_lote: fecha_actualizacion_formato	fecha_actualizacion	3, 4, 9	Normalizar texto	Conteo o valor del inventario incorrecto
inventario_lote: cantidad_invalida	cantidad	6, 10	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto
inventario_lote: producto_inexistente	id_producto	2, 5, 6, 7	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto
inventario_lote: lote_inexistente	id_lote	2, 5, 6, 7, 8, 9, 10	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto
inventario_lote: actualizacion_fuera_periodo	fecha_actualizacion	3, 4, 9	Separar registro si no es recuperable	Conteo o valor del inventario incorrecto

Las filas CSV incluyen el encabezado: la primera fila de datos es la 2. Una fila puede aparecer en más de una regla. Las fallas de relación también incluyen los registros cuyo padre fue descartado al aplicar las reglas de calidad.

VALIDACIÓN Y COMPARACIÓN ANTES Y DESPUÉS

Archivo	Iniciales	Conservados	Corregidos	Sin cambios	Descartados
producto	50	44	3	41	6
lote	50	42	2	40	8
inventario_lote	50	42	2	40	8

Archivo	Aptos antes	No aptos antes	Aptos después	No aptos después
producto	80 %	20 %	100 %	0 %
lote	78 %	22 %	100 %	0 %
inventario_lote	78 %	22 %	100 %	0 %

Conservados significa todas las filas que permanecen: corregidos más sin cambios. Se clasifica como corregido un registro conservado cuyo contenido cambió durante la limpieza. La primera copia de un duplicado permanece sin cambios y la repetición adicional se descarta. Descartados son todas las filas excluidas. Los 100 % aptos finales se calculan sobre los registros conservados, no sobre los 50 originales.

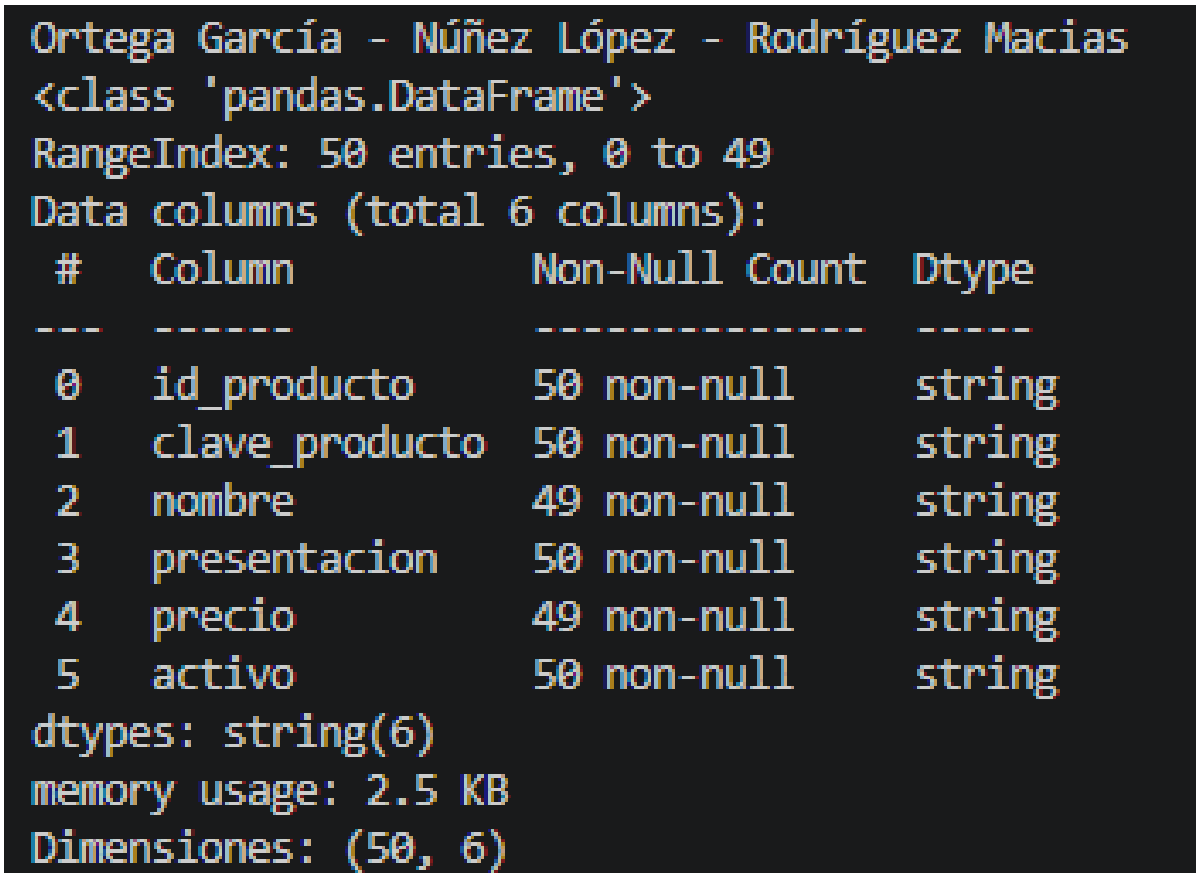
La validación final no encontró nulos obligatorios, claves repetidas, valores inválidos ni referencias rotas en los archivos exportados. Esto comprueba las reglas definidas para el ejercicio; no demuestra que los datos ficticios correspondan al inventario real de Pepsi.

EVIDENCIAS DE LA VALIDACIÓN DE LOS CSV

Las siguientes capturas muestran la inspección inicial, las reglas que detectaron errores y el resumen final de los tres archivos. En los tres casos la validación posterior a la limpieza reportó cero filas no aptas.

Figura 74

Inspección inicial del archivo producto.



Nota. Captura de ejecución proporcionada por el equipo.

Figura 75

Reglas de calidad detectadas en producto.

	id_producto	clave_producto	nombre	presentacion	precio	activo
count	50	50	49	50	49	50
unique	49	49	6	11	48	2
top	21	BEB021	7 Up	2 litros	39.72	1
freq	2	2	10	6	2	49

```
Errores por regla:
id_producto_nulo            0
id_producto_espacios        0
clave_producto_nulo         0
clave_producto_espacios     1
nombre_nulo                 1
nombre_espacios             1
presentacion_nulo           0
presentacion_espacios       0
precio_nulo                 1
precio_espacios             0
activo_nulo                 0
activo_espacios             0
duplicado_exacto            2
clave_primaria_repetida     2
id_producto_tipo            0
precio_tipo                 2
activo_tipo                 0
clave_producto_formato      1
nombre_formato              1
presentacion_formato        1
clave_producto_longitud     0
nombre_longitud             0
presentacion_longitud       0
clave_negocio_1_repetida    2
id_producto_invalido        0
precio_invalido             1
precio_decimales            0
activo_invalido             1
dtype: Int64
```

Nota. Captura de ejecución proporcionada por el equipo.

Figura 76

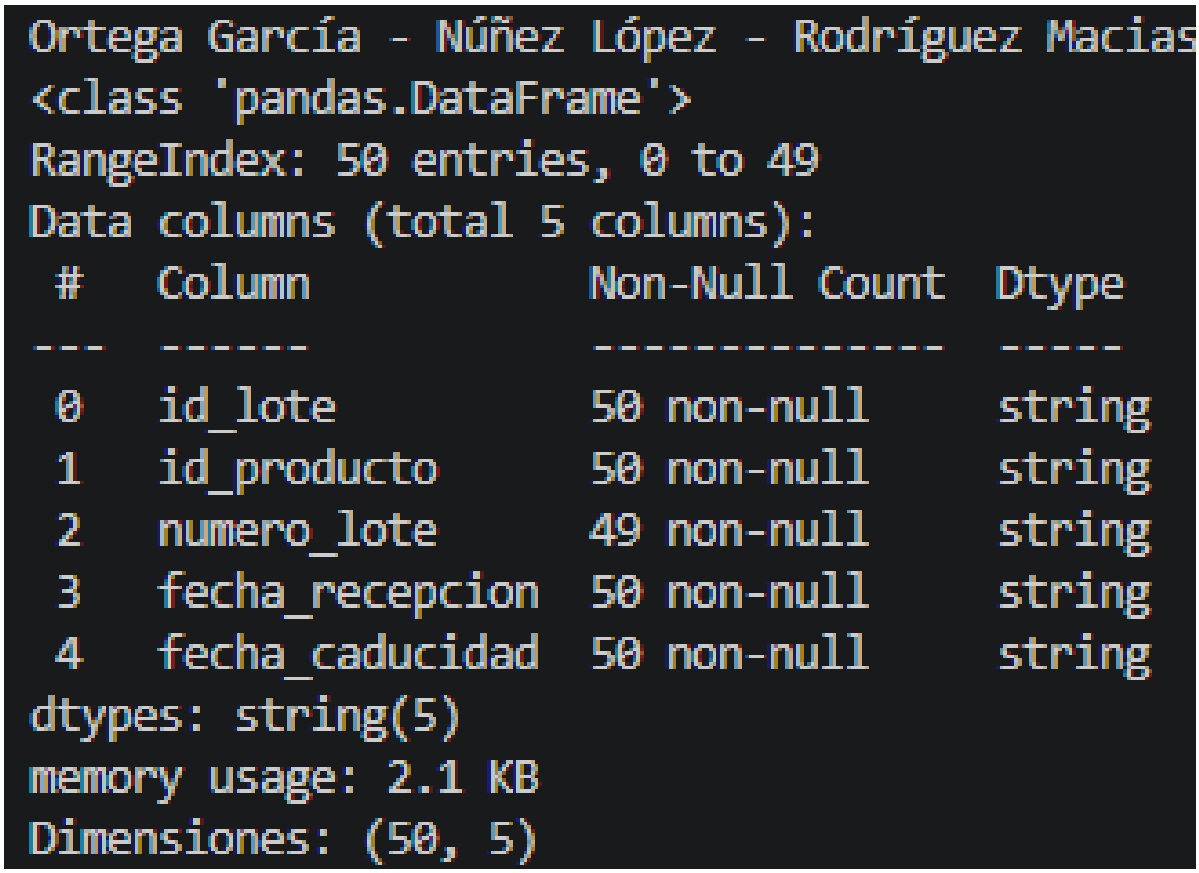
Resultado de limpieza y validación final de producto.

```
id_producto clave_producto       nombre presentacion     precio activo
          1         BEB001         <NA>       355 ml     116.11      1
          2         BEB002        Pepsi       355 ml      59.43      1
          3         BEB003         7 Up       600 ML     139.37      1
          4         BEB004     Gatorade       355 ml sin precio      1
          5         BEB005 Mountain Dew       355 ml        -25      1
          6         BEB006        Pepsi       600 ml      33.73      3
          7         BEB007      Mirinda       600 ml       <NA>      1
          8       beb008           7 Up       600 ml     185.59      1
         21         BEB021        Pepsi     2 litros      39.72      1
         21         BEB021        Pepsi     2 litros      39.72      1
Resumen de calidad:
archivo: producto
iniciales: 50
aptos_antes: 40
no_aptos_antes: 10
porcentaje_apto_antes: 80.0
porcentaje_no_apto_antes: 20.0
conservados: 44
corregidos: 3
sin_cambios: 41
descartados: 6
aptos_despues: 44
no_aptos_despues: 0
porcentaje_apto_despues: 100.0
porcentaje_no_apto_despues: 0.0
Validación final: 0 filas no aptas.
```

Nota. Captura de ejecución proporcionada por el equipo.

Figura 77

Inspección inicial del archivo lote.



Nota. Captura de ejecución proporcionada por el equipo.

Figura 78

Reglas de calidad detectadas en lote.

```
       id_lote id_producto numero_lote fecha_recepcion fecha_caducidad
count       50          50          49              50              50
unique      49          49          48              47              46
top         21          21      LOT021      2026-04-04      2027-04-04
freq         2           2           2               3               3
Errores por regla:
id_lote_nulo                0
id_lote_espacios            0
id_producto_nulo            0
id_producto_espacios        0
numero_lote_nulo            1
numero_lote_espacios        1
fecha_recepcion_nulo        0
fecha_recepcion_espacios    0
fecha_caducidad_nulo        0
fecha_caducidad_espacios    0
duplicado_exacto            2
clave_primaria_repetida     2
id_lote_tipo                0
id_producto_tipo            1
numero_lote_formato         2
numero_lote_longitud        0
clave_negocio_1_repetida    2
fecha_recepcion_invalida    2
fecha_recepcion_formato     2
fecha_caducidad_invalida    0
fecha_caducidad_formato     0
id_lote_invalido            0
id_producto_invalido        0
producto_inexistente        5
recepcion_fuera_periodo     3
caducidad_antes_recepcion   1
dtype: Int64
```

Nota. Captura de ejecución proporcionada por el equipo.

Figura 79

Resultado de limpieza y validación final de lote.

```
Filas afectadas:
id_lote    id_producto numero_lote fecha_recepcion fecha_caducidad
      1              1        <NA>       2026-07-05       2027-07-05
      2              2     LOT002        2026-01-29       2027-01-29
      3              3       lot003      2026-06-05       2027-06-05
      4              4       LOT004      15/03/2026       2027-02-21
      5 producto cinco       LOT005      2026-01-27       2027-01-27
      6            999       LOT006      2026-07-13       2027-07-13
      7              7       LOT007      2026-06-03       2025-01-01
      8              8       LOT008      2026-02-30       2027-06-18
      9              9       LOT009      2025-12-01       2027-05-16
     21             21       LOT021      2026-04-04       2027-04-04
     21             21       LOT021      2026-04-04       2027-04-04
Resumen de calidad:
archivo: lote
iniciales: 50
aptos_antes: 39
no_aptos_antes: 11
porcentaje_apto_antes: 78.0
porcentaje_no_apto_antes: 22.0
conservados: 42
corregidos: 2
sin_cambios: 40
descartados: 8
aptos_despues: 42
no_aptos_despues: 0
porcentaje_apto_despues: 100.0
porcentaje_no_apto_despues: 0.0
Validación final: 0 filas no aptas.
```

Nota. Captura de ejecución proporcionada por el equipo.

Figura 80

Inspección inicial del archivo inventario_lote.

```
Ortega García - Núñez López - Rodríguez Macias
<class 'pandas.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   id_inventario       50 non-null     string
 1   id_almacen          50 non-null     string
 2   id_lote             50 non-null     string
 3   id_producto         50 non-null     string
 4   cantidad            49 non-null     string
 5   fecha_actualizacion 50 non-null     string
dtypes: string(6)
memory usage: 2.5 KB
Dimensiones: (50, 6)
```

Nota. Captura de ejecución proporcionada por el equipo.

Figura 81

Reglas de calidad detectadas en inventario_lote.

	id_inventario	id_almacen	id_lote	id_producto	cantidad	fecha_actualizacion
count	50	50	50	50	49	50
unique	49	1	49	48	43	4
top	21	1	21	40	254	2026-10-31 12:00:00
freq	2	50	2	2	2	47

Errores por regla:

id_inventario_nulo	0
id_inventario_espacios	0
id_almacen_nulo	0
id_almacen_espacios	0
id_lote_nulo	0
id_lote_espacios	0
id_producto_nulo	0
id_producto_espacios	0
cantidad_nulo	1
cantidad_espacios	0
fecha_actualizacion_nulo	0
fecha_actualizacion_espacios	1
duplicado_exacto	2
clave_primaria_repetida	2
id_inventario_tipo	0
id_almacen_tipo	0
id_lote_tipo	0
id_producto_tipo	0
cantidad_tipo	2
clave_negocio_1_repetida	2
fecha_actualizacion_invalida	3
fecha_actualizacion_formato	3
id_inventario_invalido	0
id_almacen_invalido	0
id_lote_invalido	0
id_producto_invalido	0
cantidad_invalida	2
almacen_inexistente	0
producto_inexistente	4
lote_inexistente	7
producto_no_corresponde_lote	0
actualizacion_fuera_periodo	3

dtype: Int64

Nota. Captura de ejecución proporcionada por el equipo.

Figura 82

Resultado de limpieza y validación final de inventario_lote.

```
Filas afectadas:
id_inventario id_almacen id_lote id_producto cantidad   fecha_actualizacion
            1          1       1           1     <NA>   2026-10-31 12:00:00
            2          1       2           2       11  2026-10-31 12:00:00
            3          1       3           3      481   31/10/2026 12:00:00
            4          1       4           4   muchas   2026-10-31 12:00:00
            5          1       5           5      -12   2026-10-31 12:00:00
            6          1     999           6       86   2026-10-31 12:00:00
            7          1       7          40      312   2026-10-31 12:00:00
            8          1       8           8       59   2026-02-30 12:00:00
            9          1       9           9      1.5   2026-10-31 12:00:00
           21          1      21          21      254   2026-10-31 12:00:00
           21          1      21          21      254   2026-10-31 12:00:00
Resumen de calidad:
archivo: inventario_lote
iniciales: 50
aptos_antes: 39
no_aptos_antes: 11
porcentaje_apto_antes: 78.0
porcentaje_no_apto_antes: 22.0
conservados: 42
corregidos: 2
sin_cambios: 40
descartados: 8
aptos_despues: 42
no_aptos_despues: 0
porcentaje_apto_despues: 100.0
porcentaje_no_apto_despues: 0.0
Validación final: 0 filas no aptas.
```

Nota. Captura de ejecución proporcionada por el equipo.

EXPORTACIÓN Y REINTEGRACIÓN EN SQL SERVER

Se exportan archivos CSV con index=False para evitar que el índice interno de Pandas se convierta en una columna del almacén. Los acentos se conservan con UTF-8. Los archivos de origen permanecen separados de los resultados corregidos.

La reintegración se prepara en el esquema proyecto_u2 de pepsi_distribucion. Esto mantiene las tablas originales de dbo y evita reemplazar sus datos. Primero se debe ejecutar el código de creación; después, el script Python solicita usuario y contraseña de SQL

Server. La contraseña no se escribe en los archivos. El controlador utilizado es ODBC Driver 18 for SQL Server, disponible en Linux.

El orden es almacen, producto, lote e inventario_lote para respetar las relaciones. La carga se realiza en una transacción: si falla una inserción, no se confirma el conjunto. El programa también verifica que el destino esté vacío para evitar cargar los mismos registros dos veces (SQLAlchemy, s. f.). Las capturas recibidas del equipo confirman la carga de un almacén, 44 productos, 42 lotes y 42 inventarios. Fueron tomadas en Windows con autenticación integrada; el script de este proyecto conserva la adaptación para Linux mediante usuario y contraseña de SQL Server.

Figura 83

Listado de 00_preparar_sql.sql, parte 1

```
# Ortega García - Núñez López - Rodríguez Macias
# 00_preparar_sql.sql | parte 1

  1 -- Proyecto Unidad 2
  2 -- ? Juan Manuel Ortega García
  3 -- ! Bernardo Guillermo Núñez López
  4 -- * Carlos Gabriel Rodríguez Macias
  5
  6 -- ? Estructura transcrita de las figuras 6 y 6.1 del documento del proyecto
     1.
  7 -- * No se utilizaron los scripts locales de Unidad1 como código base.
  8 -- * Cambiamos únicamente el esquema de destino para conservar los datos de
    dbo.
  9 -- * Ejecutar este archivo una sola vez, después de revisar la base
    seleccionada.
 10 Use pepsi_distribucion;
 11 GO
 12
 13 Create Schema proyecto_u2;
 14 GO
 15
 16 -- ? Almacén de referencia. No es un cuarto CSV ni una cuarta tabla
    analizada.
 17 Create Table proyecto_u2.almacen
 18 (
 19   id_almacen int Primary Key Identity(1, 1),
 20   clave_almacen varchar(10) Not Null Unique,
 21   nombre varchar(100) Not Null,
 22   ubicacion varchar(200) Not Null,
 23   activo bit Not Null Default 1
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 84

Listado de 00_preparar_sql.sql, parte 2

```
# Ortega García - Núñez López - Rodríguez Macias
# 00_preparar_sql.sql | parte 2

 24 );
 25
 26 Create Table proyecto_u2.producto
 27 (
 28   id_producto int Primary Key Identity(1, 1),
 29   clave_producto varchar(15) Not Null Unique,
 30   nombre varchar(100) Not Null,
 31   presentacion varchar(50) Not Null,
 32   precio decimal(10, 2) Not Null,
 33   activo bit Not Null Default 1,
 34   Constraint ck_u2_producto_precio Check (precio > 0)
 35 );
 36
 37 Create Table proyecto_u2.lote
 38 (
 39   id_lote int Primary Key Identity(1, 1),
 40   id_producto int Not Null,
 41   numero_lote varchar(30) Not Null,
 42   fecha_recepcion date Not Null,
 43   fecha_caducidad date Not Null,
 44   Constraint fk_u2_lote_producto Foreign Key (id_producto)
 45     References proyecto_u2.producto(id_producto),
 46   Constraint uq_u2_lote_producto_numero Unique (id_producto, numero_lote),
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 85

Listado de 00_preparar_sql.sql, parte 3

```
# Ortega García - Núñez López - Rodríguez Macias
# 00_preparar_sql.sql | parte 3

 47   Constraint uq_u2_lote_validacion Unique (id_lote, id_producto),
 48   Constraint ck_u2_lote_fechas Check (fecha_caducidad > fecha_recepcion)
 49 );
 50
 51 Create Table proyecto_u2.inventario_lote
 52 (
 53   id_inventario int Primary Key Identity(1, 1),
 54   id_almacen int Not Null,
 55   id_lote int Not Null,
 56   id_producto int Not Null,
 57   cantidad int Not Null Default 0,
 58   fecha_actualizacion datetime2 Not Null Default Sysdatetime(),
 59   Constraint fk_u2_inventario_almacen Foreign Key (id_almacen)
 60     References proyecto_u2.almacen(id_almacen),
 61   Constraint fk_u2_inventario_lote_producto Foreign Key (id_lote,
    id_producto)
 62     References proyecto_u2.lote(id_lote, id_producto),
 63   Constraint uq_u2_inventario_almacen_lote Unique (id_almacen, id_lote),
 64   Constraint uq_u2_inventario_validacion
 65     Unique (id_inventario, id_lote, id_producto, id_almacen),
 66   Constraint ck_u2_inventario_cantidad Check (cantidad >= 0)
 67 );
 68 GO
 69
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 86

Listado de 05_cargar_sql.py, parte 1

```
# Ortega García - Núñez López - Rodríguez Macias
# 05_cargar_sql.py | parte 1

  1 # Proyecto Unidad 2
  2 # ? Juan Manuel Ortega García
  3 # ! Bernardo Guillermo Núñez López
  4 # * Carlos Gabriel Rodríguez Macias
  5
  6 # ? Usar SQLAlchemy y el controlador de SQL Server disponible en Linux.
  7 from pathlib import Path
  8 from getpass import getpass
  9 from decimal import Decimal
 10 import os
 11 import pandas as pd
 12 import pyodbc
 13 from sqlalchemy import create_engine, URL, text
 14 from sqlalchemy.exc import SQLAlchemyError
 15
 16 ruta = Path(__file__).resolve().parent.parent
 17 print('Ortega García - Núñez López - Rodríguez Macias')
 18 print('Controladores disponibles:', pyodbc.drivers())
 19 servidor = os.getenv('SQLSERVER_HOST', 'localhost,1433')
 20 base_datos = os.getenv('SQLSERVER_DATABASE', 'pepsi_distribucion')
 21 usuario = input('Usuario de SQL Server: ')
 22 contrasena = getpass('Contraseña de SQL Server: ')
 23
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 87

Listado de 05_cargar_sql.py, parte 2

```
# Ortega García - Núñez López - Rodríguez Macias
# 05_cargar_sql.py | parte 2

# * En Linux no se supone una sesión integrada de Windows.
url = URL.create(
  'mssql+pyodbc', username=usuario, password=contrasena,
  host=servidor.split(',')[0], port=int(servidor.split(',')[1]) if ',' in
servidor else 1433,
  database=base_datos,
  query={'driver': 'ODBC Driver 18 for SQL Server',
         'Encrypt': 'yes', 'TrustServerCertificate': 'yes'}
)
# * TrustServerCertificate se usa solo en esta instancia local académica.
motor = create_engine(url)

# ? Cargar las referencias antes de las tablas dependientes en una
transacción.
try:
  with motor.begin() as conexion:
    for tabla in ['producto', 'lote', 'inventario_lote', 'almacen']:
      cantidad = conexion.execute(text(f'Select Count(*) From
proyecto_u2.{tabla}')).scalar_one()
      if cantidad != 0:
        raise ValueError('El destino ya contiene datos. No se duplicará la
carga.')
    conexion.execute(text('Set Identity_Insert proyecto_u2.almacen On'))
    conexion.execute(text(
      "Insert Into proyecto_u2.almacen "
      "(id_almacen, clave_almacen, nombre, ubicacion, activo) "
      "Values (1, 'ALM001', 'Almacén Nuevo Laredo', 'Nuevo Laredo,
Tamaulipas', 1)"
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 88

Listado de 05_cargar_sql.py, parte 3

```
# Ortega García - Núñez López - Rodríguez Macias
# 05_cargar_sql.py | parte 3

 47     ))
 48     conexion.execute(text('Set Identity_Insert proyecto_u2.almacen Off'))
 49     for tabla in ['producto', 'lote', 'inventario_lote']:
 50       datos = pd.read_csv(ruta / 'csv_limpios' / f'{tabla}.csv')
 51       if datos.empty or datos.isnull().any().any():
 52         raise ValueError(f'El archivo {tabla} está vacío o contiene nulos.')
 53       if tabla == 'producto':
 54         datos['precio'] = datos['precio'].apply(lambda valor:
    Decimal(str(valor)))
 55       columnas = ', '.join(datos.columns)
 56       parametros = ', '.join(':' + columna for columna in datos.columns)
 57       instruccion = text(f'Insert Into proyecto_u2.{tabla} ({columnas})
    Values ({parametros})')
 58       conexion.execute(text(f'Set Identity_Insert proyecto_u2.{tabla} On'))
 59       conexion.execute(instruccion, datos.to_dict(orient='records'))
 60       conexion.execute(text(f'Set Identity_Insert proyecto_u2.{tabla} Off'))
 61       print(tabla, ':', len(datos), 'filas insertadas dentro de la
    transacción.')
 62   print('Transacción confirmada: carga completa en SQL Server.')
 63 except (SQLAlchemyError, ValueError, FileNotFoundError) as error:
 64   print('La carga no se confirmó:', error)
 65   raise SystemExit(1)
 66 finally:
 67   motor.dispose()
 68
```

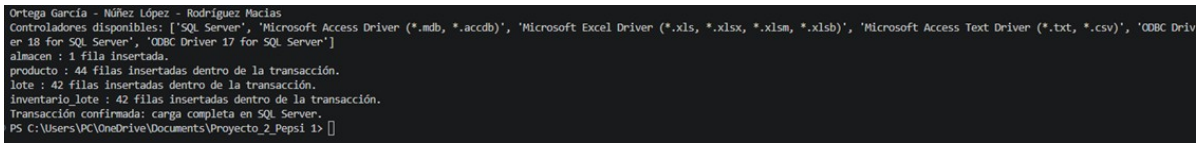
Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

EVIDENCIAS DE CARGA Y CONSULTAS EN SQL SERVER

La salida de carga confirma la transacción y los conteos de filas exportadas. Las consultas mostradas después verifican que los registros limpios quedaron disponibles en las tablas producto, lote e inventario_lote del esquema proyecto_u2.

Figura 89

Carga confirmada de los datos limpios en SQL Server.



Nota. Captura de ejecución proporcionada por el equipo.

Figura 90

Consulta de registros cargados en producto.

	id_producto	clave_producto	nombre	presentacion	precio	activo
1	2	BEB002	Pepsi	355 ml	59.43	1
2	3	BEB003	7 Up	600 ml	139.37	1
3	8	BEB008	7 Up	600 ml	185.59	1
4	9	BEB009	Gatorade	600 ml	40.84	1
5	10	BEB010	Mountain Dew	600 ml	129.82	1
6	11	BEB011	Pepsi	1 litro	200.96	1
7	12	BEB012	Mirinda	1 litro	29.00	1
8	13	BEB013	7 Up	1 litro	176.27	1
9	14	BEB014	Gatorade	1 litro	80.35	1
10	15	BEB015	Mountain Dew	1 litro	22.28	1

Nota. Captura de ejecución proporcionada por el equipo.

Figura 91

Consulta de registros cargados en lote.

	id_lote	id_producto	numero_lote	fecha_recepcion	fecha_caducidad
1	2	2	LOT002	2026-01-29	2027-01-29
2	3	3	LOT003	2026-06-05	2027-06-05
3	10	10	LOT010	2026-05-25	2027-05-25
4	11	11	LOT011	2026-07-31	2027-07-31
5	12	12	LOT012	2026-05-07	2027-05-07
6	13	13	LOT013	2026-09-09	2027-09-09
7	14	14	LOT014	2026-04-09	2027-04-09
8	15	15	LOT015	2026-03-09	2027-03-09
9	16	16	LOT016	2026-02-18	2027-02-18
10	17	17	LOT017	2026-08-01	2027-08-01
11	18	18	LOT018	2026-01-23	2027-01-23
12	19	19	LOT019	2026-03-23	2027-03-23
13	20	20	LOT020	2026-05-15	2027-05-15
14	21	21	LOT021	2026-04-04	2027-04-04

Nota. Captura de ejecución proporcionada por el equipo.

Figura 92

Consulta de registros cargados en inventario_lote.

	id_inventario	id_almacen	id_lote	id_producto	cantidad	fecha_actualizacion
1	2	1	2	2	11	2026-10-31 12:00:00.0000000
2	3	1	3	3	481	2026-10-31 12:00:00.0000000
3	10	1	10	10	30	2026-10-31 12:00:00.0000000
4	11	1	11	11	111	2026-10-31 12:00:00.0000000
5	12	1	12	12	393	2026-10-31 12:00:00.0000000
6	13	1	13	13	147	2026-10-31 12:00:00.0000000
7	14	1	14	14	66	2026-10-31 12:00:00.0000000
8	15	1	15	15	378	2026-10-31 12:00:00.0000000
9	16	1	16	16	126	2026-10-31 12:00:00.0000000
10	17	1	17	17	203	2026-10-31 12:00:00.0000000
11	18	1	18	18	200	2026-10-31 12:00:00.0000000
12	19	1	19	19	469	2026-10-31 12:00:00.0000000
13	20	1	20	20	446	2026-10-31 12:00:00.0000000
14	21	1	21	21	254	2026-10-31 12:00:00.0000000
15	22	1	22	22	41	2026-10-31 12:00:00.0000000
16	23	1	23	23	85	2026-10-31 12:00:00.0000000
17	24	1	24	24	229	2026-10-31 12:00:00.0000000
18	25	1	25	25	205	2026-10-31 12:00:00.0000000
19	26	1	26	26	281	2026-10-31 12:00:00.0000000
20	27	1	27	27	142	2026-10-31 12:00:00.0000000

Nota. Captura de ejecución proporcionada por el equipo.

CONSULTAS Y EFECTO DE LOS DATOS INCORRECTOS

La primera consulta suma unidades y calcula el valor del inventario por producto. Un precio negativo o una existencia duplicada habría alterado ese valor. La segunda identifica lotes próximos a caducar: una fecha mal interpretada cambia la prioridad de despacho. La tercera calcula promedio, mínimo y máximo de precios; un precio inválido puede distorsionar estos indicadores o impedir el cálculo.

TABLA TEMPORAL Y VISTA ENCRIPTADA

La tabla temporal prepara un resumen para revisar existencias durante la misma conexión, sin modificar los registros del almacén. Puede utilizarse para organizar la

revisión de cantidades y valores antes de un reporte. La vista reúne la consulta de existencias para reutilizarla.

WITH ENCRYPTION oculta la definición de la vista en las consultas habituales del catálogo; no cifra los datos que devuelve ni sustituye los permisos. Debe conservarse el código fuente para mantenimiento y se deben revisar sus limitaciones de publicación y replicación (Microsoft, s. f.-a).

Figura 93

Listado de 06_consultas.sql, parte 1

```
# Ortega García - Núñez López - Rodríguez Macias
# 06_consultas.sql | parte 1

  1 -- Proyecto Unidad 2
  2 -- ? Juan Manuel Ortega García
  3 -- ! Bernardo Guillermo Núñez López
  4 -- * Carlos Gabriel Rodríguez Macias
  5 Use pepsi_distribucion;
  6 GO
  7
  8 -- ? 1 Revisar el valor del inventario por producto.
  9 Select p.id_producto, p.nombre, p.presentacion,
 10   Sum(i.cantidad) As unidades,
 11   Sum(i.cantidad * p.precio) As valor_inventario
 12 From proyecto_u2.producto p
 13 Inner Join proyecto_u2.inventario_lote i On p.id_producto = i.id_producto
 14 Where p.activo = 1 And i.cantidad > 0
 15 Group By p.id_producto, p.nombre, p.presentacion
 16 Order By valor_inventario Desc;
 17
 18 -- ? 2 Revisar lotes con caducidad próxima desde el corte del ejercicio.
 19 Select p.nombre, l.numero_lote, l.fecha_caducidad,
 20   Sum(i.cantidad) As unidades_por_caducar,
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 94

Listado de 06_consultas.sql, parte 2

```
# Ortega García - Núñez López - Rodríguez Macias
# 06_consultas.sql | parte 2

 21   Min(l.fecha_caducidad) As primera_caducidad
 22 From proyecto_u2.lote l
 23 Inner Join proyecto_u2.producto p On l.id_producto = p.id_producto
 24 Inner Join proyecto_u2.inventario_lote i
 25   On l.id_lote = i.id_lote And l.id_producto = i.id_producto
 26 Where l.fecha_caducidad >= '2026-11-02' And l.fecha_caducidad < '2027-03-01'
 27 Group By p.nombre, l.numero_lote, l.fecha_caducidad
 28 Order By l.fecha_caducidad Asc;
 29
 30 -- ? 3 Comparar precios de productos activos.
 31 Select nombre, Count(*) As presentaciones,
 32   Avg(precio) As precio_promedio, Min(precio) As precio_minimo,
 33   Max(precio) As precio_maximo
 34 From proyecto_u2.producto
 35 Where activo = 1 And precio > 0
 36 Group By nombre
 37 Order By precio_promedio Desc;
 38
 39 -- ? Crear una tabla temporal para preparar la revisión del almacén.
 40 -- * La tabla temporal solo se utiliza durante esta conexión.
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 95

Listado de 06_consultas.sql, parte 3

```
# Ortega García - Núñez López - Rodríguez Macias
# 06_consultas.sql | parte 3

 41 Create Table #revision_inventario
 42 (
 43   id_producto int,
 44   nombre varchar(100),
 45   unidades int,
 46   valor_inventario decimal(18, 2)
 47 );
 48
 49 Insert Into #revision_inventario
 50 Select p.id_producto, p.nombre, Sum(i.cantidad), Sum(i.cantidad * p.precio)
 51 From proyecto_u2.producto p
 52 Inner Join proyecto_u2.inventario_lote i On p.id_producto = i.id_producto
 53 Group By p.id_producto, p.nombre;
 54
 55 Select * From #revision_inventario
 56 Where unidades > 0
 57 Order By valor_inventario Desc;
 58 GO
 59
 60 -- ? Crear la vista encriptada para la consulta de existencias.
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

Figura 96

Listado de 06_consultas.sql, parte 4

```
# Ortega García - Núñez López - Rodríguez Macias
# 06_consultas.sql | parte 4

 61 -- * With Encryption oculta la definición; no cifra las filas ni reemplaza
    permisos.
 62 Create View proyecto_u2.vw_existencias_equipo
 63 With Encryption
 64 As
 65 Select p.id_producto, p.nombre, p.presentacion, p.precio,
 66   Sum(i.cantidad) As existencia,
 67   Sum(i.cantidad * p.precio) As valor_inventario
 68 From proyecto_u2.producto p
 69 Inner Join proyecto_u2.inventario_lote i On p.id_producto = i.id_producto
 70 Group By p.id_producto, p.nombre, p.presentacion, p.precio;
 71 GO
 72
 73 Select * From proyecto_u2.vw_existencias_equipo
 74 Order By valor_inventario Desc;
 75
 76 -- ? Comprobar que SQL Server no expone el texto de la vista.
 77 Select Objectproperty(Object_Id('proyecto_u2.vw_existencias_equipo'),
    'IsEncrypted')
 78   As vista_encriptada;
 79 GO
 80
```

Nota. Imagen del archivo de código; los comentarios identifican a los integrantes.

El archivo contiene tres consultas con WHERE, ORDER BY y agregaciones, además de la tabla temporal y la vista. Las evidencias siguientes muestran la ejecución de las consultas, de la tabla temporal y de la vista encriptada.

Figura 97

Resultado de la consulta de lotes próximos a caducar.

```sql
--JUAN MANUEL ORTEGA GARCIA
--BERNARDO GUILLERMO NUÑEZ LOPEZ
--CARLOS GABRIEL RODRIGUEZ MACIAS
--ATHALA DULCE EDITH GÓMEZ ARELLANO

SELECT
    Min(lote.fecha_caducidad) AS primera_caducidad
FROM proyecto_u2.lote AS lote
INNER JOIN proyecto_u2.producto AS producto ON lote.id_producto = producto.id_producto
INNER JOIN proyecto_u2.inventario_lote AS inventario_lote
    ON lote.id_lote = inventario_lote.id_lote AND lote.id_producto = inventario_lote.id_producto
WHERE lote.fecha_caducidad >= '2026-11-02' AND lote.fecha_caducidad < '2027-03-01'
GROUP BY producto.nombre, lote.numero_lote, lote.fecha_caducidad
ORDER BY lote.fecha_caducidad ASC;
```

90 % 1 0

Resultados Mensajes

	primera_caducidad
1	2027-01-18
2	2027-01-19
3	2027-01-20
4	2027-01-22
5	2027-01-23
6	2027-01-26
7	2027-01-29
8	2027-02-14
9	2027-02-18

Nota. Captura de ejecución proporcionada por el equipo.

Figura 98

Resultado de la consulta de comparación de precios por producto.

```sql
--JUAN MANUEL ORTEGA GARCIA
--BERNARDO GUILLERMO NUÑEZ LOPEZ
--CARLOS GABRIEL RODRIGUEZ MACIAS
--ATHALA DULCE EDITH GÓMEZ ARELLANO

SELECT
    nombre,
    Count(*) AS presentaciones,
    Avg(precio) AS precio_promedio,
    Min(precio) AS precio_minimo,
    Max(precio) AS precio_maximo
FROM proyecto_u2.producto
WHERE activo = 1 AND precio > 0
GROUP BY nombre
ORDER BY precio_promedio DESC;

----------------------------------------------------------------------
```

90 % 1 0

Resultados Mensajes

	nombre	presentaciones	precio_promedio	precio_minimo	precio_maximo
1	Mountain Dew	8	169.492500	22.28	291.30
2	7 Up	10	136.328000	25.26	199.10
3	Gatorade	9	116.132222	29.36	216.64
4	Mirinda	8	104.610000	29.00	197.07
5	Pepsi	9	95.893333	26.24	201.03

Nota. Captura de ejecución proporcionada por el equipo.

Figura 99

Ejecución de la creación y carga de la tabla temporal de revisión.

```
--JUAN MANUEL ORTEGA GARCIA
--BERNARDO GUILLERMO NUÑEZ LOPEZ
--CARLOS GABRIEL RODRIGUEZ MACIAS
--ATHALA DULCE EDITH GÓMEZ ARELLANO

CREATE TABLE #revision_inventario
(
    id_producto INT,
    nombre VARCHAR(100),
    unidades INT,
    valor_inventario DECIMAL(18, 2)
);

INSERT INTO #revision_inventario
SELECT
    producto.id_producto,
    producto.nombre,
    Sum(inventario_lote.cantidad),
    Sum(inventario_lote.cantidad * producto.precio)
FROM proyecto_u2.producto AS producto
INNER JOIN proyecto_u2.inventario_lote AS inventario_lote
    ON producto.id_producto = inventario_lote.id_producto
GROUP BY producto.id_producto, producto.nombre;

SELECT * FROM #revision_inventario
WHERE unidades > 0
```

90 % 1 0

Mensajes

```
(42 filas afectadas)

Hora de finalización: 2026-11-06T16:17:45.8597942-06:00
```

Nota. Captura de ejecución proporcionada por el equipo.

Figura 100

Resultado de la tabla temporal ordenada por valor de inventario.

```
--JUAN MANUEL ORTEGA GARCIA
--BERNARDO GUILLERMO NUÑEZ LOPEZ
--CARLOS GABRIEL RODRIGUEZ MACIAS
--ATHALA DULCE EDITH GÓMEZ ARELLANO



SELECT * FROM #revision_inventario
WHERE unidades > 0
ORDER BY valor_inventario DESC;
```

90 % 4 0

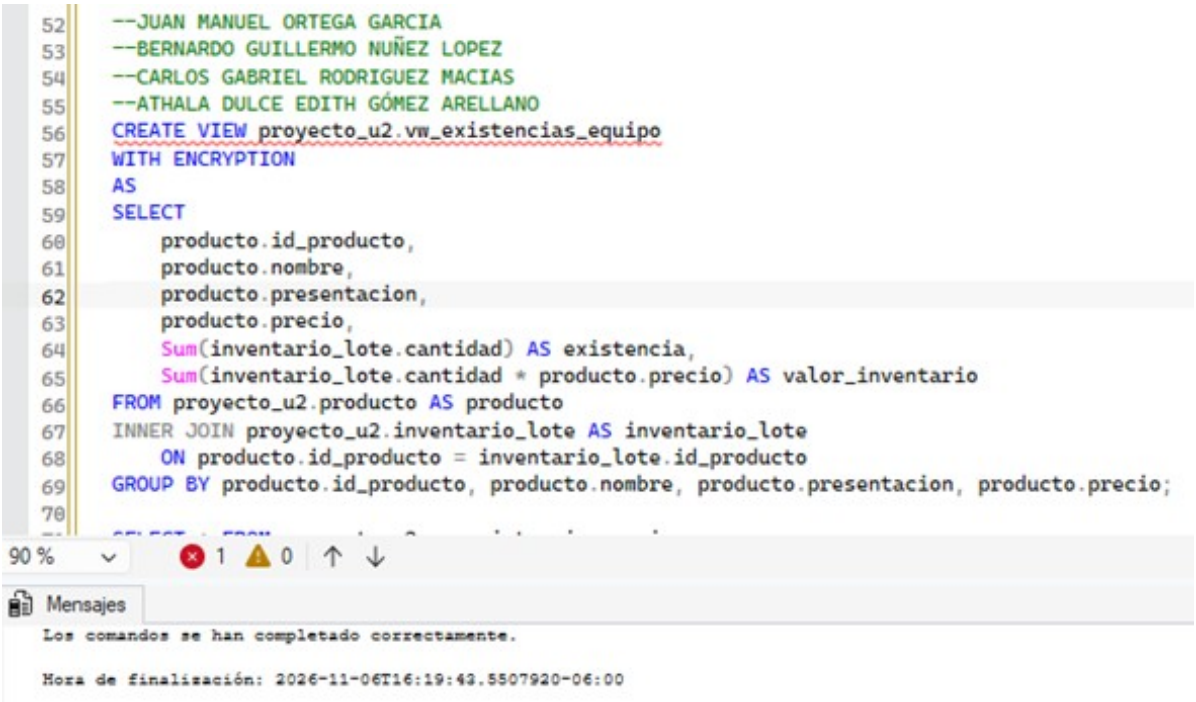
Resultados | Mensajes

	id_producto	nombre	unidades	valor_inventario
1	30	Mountain Dew	419	90332.21
2	39	Gatorade	452	86964.80
3	3	7 Up	481	67036.97
4	25	Mountain Dew	205	57592.70
5	40	Mountain Dew	194	56512.20
6	33	7 Up	281	55947.10
7	26	Pepsi	281	54873.68
8	35	Mountain Dew	361	50532.78
9	31	Pepsi	220	44226.60
10	20	Mountain Dew	446	39631.56
11	28	7 Up	452	37583.80
12	48	7 Up	337	37433.96
13	17	Mirinda	203	30874.27
14	18	7 Up	200	29404.00
15	34	Gatorade	142	28665.54
16	41	Pepsi	490	26278.70
17	13	7 Up	147	25911.69
18	47	Mirinda	118	23254.26
19	49	Gatorade	119	23036.02
20	11	Pepsi	111	22306.56

Nota. Captura de ejecución proporcionada por el equipo.

Figura 101

Creación correcta de la vista encriptada de existencias.



Nota. Captura de ejecución proporcionada por el equipo.

Figura 102

Resultado de la consulta realizada a la vista encriptada.

```
71  --JUAN MANUEL ORTEGA GARCIA
72  --BERNARDO GUILLERMO NUÑEZ LOPEZ
73  --CARLOS GABRIEL RODRIGUEZ MACIAS
74  --ATHALA DULCE EDITH GÓMEZ ARELLANO
75
76  SELECT * FROM proyecto_u2.vw_existencias_equipo
77  ORDER BY valor_inventario DESC;
78
```

90 % 1 0

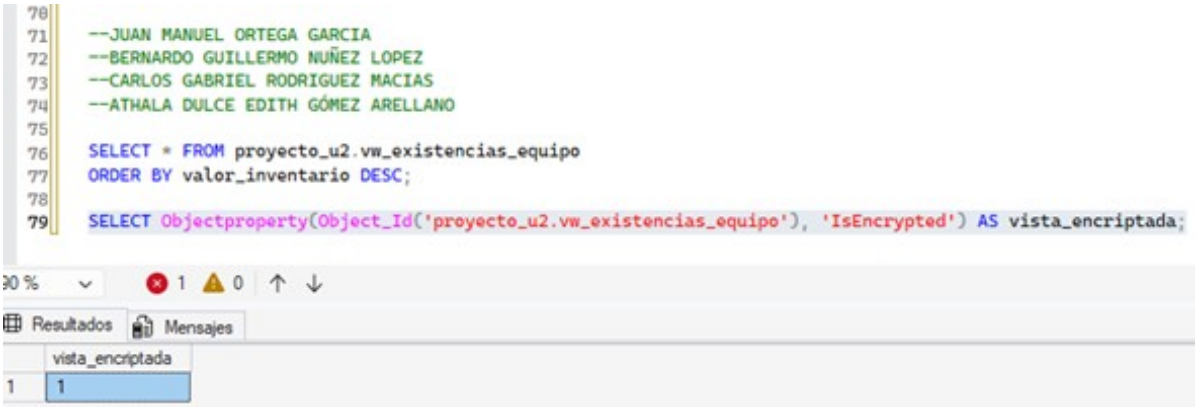
Resultados Mensajes

	id_producto	nombre	presentacion	precio	existencia	valor_inventario
1	30	Mountain Dew	3 litros	215.59	419	90332.21
2	39	Gatorade	12 latas	192.40	452	86964.80
3	3	7 Up	600 ml	139.37	481	67036.97
4	25	Mountain Dew	2 litros	280.94	205	57592.70
5	40	Mountain Dew	12 latas	291.30	194	56512.20
6	33	7 Up	6 latas	199.10	281	55947.10
7	26	Pepsi	3 litros	195.28	281	54873.68
8	35	Mountain Dew	6 latas	139.98	361	50532.78
9	31	Pepsi	6 latas	201.03	220	44226.60
10	20	Mountain Dew	1.5 litros	88.86	446	39631.56
11	28	7 Up	3 litros	83.15	452	37583.80
12	48	7 Up	12 botellas	111.08	337	37433.96
13	17	Mirinda	1.5 litros	152.09	203	30874.27
14	18	7 Up	1.5 litros	147.02	200	29404.00
15	34	Gatorade	6 latas	201.87	142	28665.54
16	41	Pepsi	24 latas	53.63	490	26278.70
17	13	7 Up	1 litro	176.27	147	25911.69
18	47	Mirinda	12 botellas	197.07	118	23254.26
19	49	Gatorade	12 botellas	193.58	119	23036.02
20	11	Pepsi	1 litro	200.96	111	22306.56

Nota. Captura de ejecución proporcionada por el equipo.

Figura 103

Comprobación de que la vista se encuentra encriptada.



Nota. Captura de ejecución proporcionada por el equipo.

CONCLUSIÓN ACTUALIZADA

Con este proyecto relacionamos la administración de la base de datos con la calidad de la información que recibe. Python permitió generar registros, encontrar fallas y corregir únicamente lo que podía comprobarse. La validación evitó aceptar cantidades, precios o relaciones incorrectas que afectarían las decisiones del almacén.

El proceso ETL permitió conservar el origen, transformar los datos y preparar su reintegración. También comprendimos que un archivo con menos errores no basta: debemos revisar nuevamente sus reglas, justificar los descartes y comprobar sus relaciones. La carga en SQL Server y las consultas completan el proceso cuando se ejecutan y verifican con sus evidencias.

REFERENCIAS DE LA UNIDAD II

Faker. (s. f.). Welcome to Faker’s documentation. https://faker.readthedocs.io/en/master/

Microsoft. (s. f.-a). CREATE VIEW (Transact-SQL). Microsoft Learn. https://learn.microsoft.com/en-us/sql/t-sql/statements/create-view-transact-sql

Microsoft. (s. f.-b). Extract, transform, load (ETL). Microsoft Learn. https://learn.microsoft.com/azure/architecture/data-guide/relational-data/etl

pandas. (s. f.-a). pandas.to_datetime. https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html

pandas. (s. f.-b). pandas.to_numeric. https://pandas.pydata.org/docs/reference/api/pandas.to_numeric.html

SQLAlchemy. (s. f.). Working with engines and connections. https://docs.sqlalchemy.org/en/20/core/connections.html

ANEXO INTERNO PARA REVISIÓN DEL EQUIPO

Se corrigió la matriz de grupos de archivos: pedido y detalle_pedido pertenecen a PRIMARY; lote a SECUNDARIO, de acuerdo con las capturas.

Las figuras de Unidad I son evidencias históricas del documento recibido y muestran Windows/SSMS. La ejecución de Unidad II se realiza en Linux; no representan una nueva prueba local.

Revisar la afirmación del trigger de inventario: las capturas mostradas no incluyen su CREATE TRIGGER. No se debe dar por comprobada esa creación sin el código o su consulta de verificación.

La captura de detalle_entrega valida cantidades no negativas, pero no incluye cantidad_danada <= cantidad_entregada. Agregar el CHECK en otra evidencia si se mantiene esa afirmación.

Revisar antes de entregar la versión/edición de SQL Server, horarios y RPO/RTO del documento anterior. Las metas propuestas no prueban por sí solas que existan trabajos programados.

Los listados de código se generan desde los archivos reales. Las evidencias de ejecución agregadas corresponden a capturas del equipo; conservar los comentarios con los apellidos en las futuras capturas del editor.

La evidencia de carga se obtuvo en Windows con Trusted Connection. El script entregado usa autenticación de SQL Server en Linux; no se debe afirmar que la captura comprueba por sí sola esa configuración local.

Pendiente: capturar los resultados de las tres consultas con filtros y agregaciones, de la tabla temporal y de la vista encriptada en SQL Server.